算法板子合集

代码模板 / 算法笔记

生成日期：2026年3月

包含语法高亮，按文件夹分类

Data\PBDS

Data\PBDS\PBDS.cpp

```cpp
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
// include 放到前面，否则会出现未定义的引用错误

template<typename T>
using ordered_set = tree<
    T,
    null_type,
    less<T>,
    rb_tree_tag,
    tree_order_statistics_node_update
>;
//ordered_multiset<pair<int, int>> == multiset;
class PBDSMultiSet {
private:
    using pii = pair<int, int>;
    using Tree = tree<pii, null_type, less<pii>, rb_tree_tag, tree_order_statistics_node_update>;

    Tree ms;
    int uid = 0;  // 用于区分重复元素

public:
    void insert(int x) {
        ms.insert({x, uid++});
    }

    void erase(int x) {
        auto it = ms.lower_bound({x, 0});
        if (it != ms.end() && it->first == x) {
            ms.erase(it);
        }
    }

    int count(int x) {
        return ms.order_of_key({x + 1, 0}) - ms.order_of_key({x, 0});
    }

    int size() const {
        return ms.size();
    }

    int kth(int k) {  // k 从 0 开始
        if (k < 0 || k >= (int)ms.size()) return -1;
        return ms.find_by_order(k)->first;
    }

    int rank(int x) {  // 小于 x 的数量
        return ms.order_of_key({x, 0});
    }

    bool empty() const {
        return ms.empty();
    }

    void clear() {
        ms.clear();
        uid = 0;
    }
};

//map
template<typename Key, typename Value>
class PBDSMap {
private:
    using omap = tree<
        Key, Value, less<Key>,
        rb_tree_tag,
        tree_order_statistics_node_update
    >;

    omap mp;

public:
    Value& operator[](const Key& k) {
        return mp[k];  // 自动插入默认值
    }

    bool contains(const Key& k) const {
        return mp.find(k) != mp.end();
```

```
    }

    void erase(const Key& k) {
        mp.erase(k);
    }

    int size() const {
        return mp.size();
    }

    int rank(const Key& k) const {
        return mp.order_of_key(k);
    }

    pair<Key, Value> kth(int k) const {
        if (k < 0 || k >= (int)mp.size()) throw out_of_range("k out of range");
        auto it = mp.find_by_order(k);
        return {it->first, it->second};
    }

    void clear() {
        mp.clear();
    }
};

/*
https://codeforces.com/problemset/problem/1915/F
| 功能       | 接口                   | 复杂度           |
| -------- | ------------------- | ------------- |
| 插入元素     | `insert(x)`          | log n |
| 删除元素     | `erase(x)`-'erase(lower_bound(x))'           | log n |
| 第 k 小     | `*find_by_order(k)` | log n |
| 小于 x 的数量 | `order_of_key(x)`    | log n |
| 支持重复元素   | 插入 `pair<val, id>`  | log n |
| 查询元素排名   | `order_of_key(x)`    | log n |

*/
void multiset_test() {
    PBDSMultiSet s;

    s.insert(5);
    s.insert(5);
    s.insert(3);
    s.insert(7);

    cout << "Count of 5: " << s.count(5) << "\n";       // 2
    cout << "Rank of 5: " << s.rank(5) << "\n";         // 1
    cout << "1st smallest: " << s.kth(1) << "\n";       // 5
    s.erase(5);
    cout << "Count of 5 after erase: " << s.count(5) << "\n"; // 1
}
void map_test() {
    PBDSMap<int, string> mymap;

    mymap[10] = "ten";
    mymap[5] = "five";
    mymap[8] = "eight";

    cout << mymap.rank(9) << "\n";          // 输出 2（5, 8）
    auto [k, v] = mymap.kth(1);              // 第 1 小的 key
    cout << k << " => " << v << "\n";       // 输出 8 => eight

    if (mymap.contains(10)) {
        cout << "10 exists\n";
    }

}
void slu() {
    multiset_test();
}

signed main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)slu();
    return 0;
}
```

Tree\AVL

Tree\AVL\AVL.cpp

```cpp
constexpr int MAXN = 1e5 + 10;

struct AVL {
    struct Node {
        int val;      // 当前节点值
        int height;   // 当前节点高度
        int size;     // 当前子树大小
        int l, r;     // 左右儿子在数组中的编号
    } tr[MAXN];

    int root = 0, idx = 0;

    // 新建一个节点
    int new_node(int val) {
        tr[++idx] = {val, 1, 1, 0, 0};
        return idx;
    }

    // 获取节点高度
    int height(int p) {
        return p ? tr[p].height : 0;
    }

    // 获取子树大小
    int size(int p) {
        return p ? tr[p].size : 0;
    }

    // 更新节点高度和大小
    void pull(int p) {
        tr[p].height = max(height(tr[p].l), height(tr[p].r)) + 1;
        tr[p].size = size(tr[p].l) + size(tr[p].r) + 1;
    }

    // 平衡因子
    int bf(int p) {
        return height(tr[p].l) - height(tr[p].r);
    }

    // 右旋操作
    int rotate_right(int p) {
        int q = tr[p].l;
        tr[p].l = tr[q].r;
        tr[q].r = p;
        pull(p);
        pull(q);
        return q;
    }

    // 左旋操作
    int rotate_left(int p) {
        int q = tr[p].r;
        tr[p].r = tr[q].l;
        tr[q].l = p;
        pull(p);
        pull(q);
        return q;
    }

    // 平衡处理
    int maintain(int p) {
        if (bf(p) == 2) {
            if (bf(tr[p].l) < 0) {
                tr[p].l = rotate_left(tr[p].l);
            }
            p = rotate_right(p);
        } else if (bf(p) == -2) {
            if (bf(tr[p].r) > 0) {
                tr[p].r = rotate_right(tr[p].r);
            }
            p = rotate_left(p);
        } else {
            pull(p);
        }
        return p;
    }

    // 插入 val
    int insert(int p, int val) {
        if (!p) return new_node(val);
```

```
        if (val < tr[p].val) {
            tr[p].l = insert(tr[p].l, val);
        } else {
            tr[p].r = insert(tr[p].r, val);
        }
        return maintain(p);
    }

    void insert(int val) {
        root = insert(root, val);
    }

    // 获取中序最小值
    int get_min(int p) {
        while (tr[p].l) p = tr[p].l;
        return p;
    }

    // 删除 val
    int erase(int p, int val) {
        if (!p) return 0;
        if (val == tr[p].val) {
            if (!tr[p].l || !tr[p].r) {
                return tr[p].l + tr[p].r;
            } else {
                int q = get_min(tr[p].r);
                tr[p].val = tr[q].val;
                tr[p].r = erase(tr[p].r, tr[q].val);
            }
        } else if (val < tr[p].val) {
            tr[p].l = erase(tr[p].l, val);
        } else {
            tr[p].r = erase(tr[p].r, val);
        }
        return maintain(p);
    }

    void erase(int val) {
        root = erase(root, val);
    }

    // 查询比 x 小的元素个数（PBDS: order_of_key）
    int order_of_key(int x) {
        int p = root, ans = 0;
        while (p) {
            if (x <= tr[p].val) {
                p = tr[p].l;
            } else {
                ans += size(tr[p].l) + 1;
                p = tr[p].r;
            }
        }
        return ans;
    }

    // 查询第 k 小（0-based）（PBDS: find_by_order）
    int find_by_order(int k) {
        int p = root;
        while (p) {
            int left = size(tr[p].l);
            if (k < left) {
                p = tr[p].l;
            } else if (k == left) {
                return tr[p].val;
            } else {
                k -= left + 1;
                p = tr[p].r;
            }
        }
        return -1; // 不存在
    }

    // 查询 x 的前驱（小于 x 的最大值）
    int prev(int x) {
        int p = root, ans = -1;
        while (p) {
            if (tr[p].val < x) {
                ans = tr[p].val;
                p = tr[p].r;
            } else {
                p = tr[p].l;
            }
        }
        return ans;
```

```
    }

    // 查询 x 的后继（大于 x 的最小值）
    int next(int x) {
        int p = root, ans = -1;
        while (p) {
            if (tr[p].val > x) {
                ans = tr[p].val;
                p = tr[p].l;
            } else {
                p = tr[p].r;
            }
        }
        return ans;
    }
};
/*
AVL 树接口说明：

构造函数：
    AVL(int max_nodes)       // 指定最大节点数，动态分配空间

主要操作：
    void insert(int x)       // 插入值 x（支持重复）
    void erase(int x)        // 删除值 x（存在则删除一个）

查询操作：
    int order_of_key(int x)  // 返回严格小于 x 的元素个数 == rank - 1
    int find_by_order(int k) // 返回第 k 个（0-based）元素的值，越界返回 -1

辅助操作：
    int predecessor(int x)   // 返回严格小于 x 的最大值，若不存在返回 -1
    int successor(int x)     // 返回严格大于 x 的最小值，若不存在返回 -1
*/
```

Tree\AVL\示例.cpp

```cpp
#include "bits/stdc++.h"
/*
关注b站<Feijoa_Li>谢谢喵
UID:248614274
https://space.bilibili.com/248614274
 */
using namespace std;
using u64 = unsigned long long;
using u32 = unsigned;
using u128 = unsigned __int128;
using i128 = __int128;

#define int long long
#define Ist first
#define Ind second
#define itn long long
#define O (int)0
#define endl "\n"
#define PII pair<int,int>
const int inf = 1e18;


struct AVL {
    struct Node {
        int val;    // 节点值
        int siz;    // 子树大小
        int height; // 高度
        int l, r;   // 左右子节点在数组中的下标
    };

    vector<Node> tr;
    int root = 0, tot = 0;

    AVL(int max_nodes) { // 构造函数，动态设定空间大小
        tr.resize(max_nodes + 1); // 节点编号从 1 开始
    }

    // 创建新节点
    int new_node(int val) {
        tr[++tot] = {val, 1, 1, 0, 0};
        return tot;
    }

    // 获取节点高度
    int height(int p) {
        return p ? tr[p].height : 0;
    }

    // 获取子树大小
    int size(int p) {
        return p ? tr[p].siz : 0;
    }

    // 更新节点信息（大小、高度）
    void push_up(int p) {
        tr[p].siz = size(tr[p].l) + size(tr[p].r) + 1;
        tr[p].height = max(height(tr[p].l), height(tr[p].r)) + 1;
    }

    // 左旋操作
    void rotate_left(int &p) {
        int q = tr[p].r;
        tr[p].r = tr[q].l;
        tr[q].l = p;
        push_up(p);
        push_up(q);
        p = q;
    }

    // 右旋操作
    void rotate_right(int &p) {
        int q = tr[p].l;
        tr[p].l = tr[q].r;
        tr[q].r = p;
        push_up(p);
        push_up(q);
        p = q;
    }

    // 平衡操作
```

```
    void balance(int &p) {
        if (!p) return;
        int balance_factor = height(tr[p].l) - height(tr[p].r);
        if (balance_factor == 2) {
            if (height(tr[tr[p].l].l) < height(tr[tr[p].l].r)) {
                rotate_left(tr[p].l);
            }
            rotate_right(p);
        } else if (balance_factor == -2) {
            if (height(tr[tr[p].r].r) < height(tr[tr[p].r].l)) {
                rotate_right(tr[p].r);
            }
            rotate_left(p);
        } else {
            push_up(p);
        }
    }

    // 插入值 x 到以 p 为根的子树中
    void insert(int &p, int x) {
        if (!p) {
            p = new_node(x);
            return;
        }
        if (x < tr[p].val) insert(tr[p].l, x);
        else insert(tr[p].r, x);
        balance(p);
    }

    void insert(int x) {
        insert(root, x);
    }

    // 删除值 x 从以 p 为根的子树中
    void erase(int &p, int x) {
        if (!p) return;
        if (x < tr[p].val) {
            erase(tr[p].l, x);
        } else if (x > tr[p].val) {
            erase(tr[p].r, x);
        } else {
            if (!tr[p].l || !tr[p].r) {
                p = tr[p].l + tr[p].r;
            } else {
                int q = tr[p].r;
                while (tr[q].l) q = tr[q].l;
                tr[p].val = tr[q].val;
                erase(tr[p].r, tr[q].val);
            }
        }
        if (p) balance(p);
    }

    void erase(int x) {
        erase(root, x);
    }

    // order_of_key(x): 查询严格小于 x 的个数
    int order_of_key(int p, int x) {
        if (!p) return 0;
        if (x <= tr[p].val) {
            return order_of_key(tr[p].l, x);
        } else {
            return size(tr[p].l) + 1 + order_of_key(tr[p].r, x);
        }
    }

    int order_of_key(int x) {
        return order_of_key(root, x);
    }

    // find_by_order(k): 查询第 k 个（0-based）数
    int find_by_order(int p, int k) {
        if (!p) return -1;
        if (k < size(tr[p].l)) {
            return find_by_order(tr[p].l, k);
        } else if (k == size(tr[p].l)) {
            return tr[p].val;
        } else {
            return find_by_order(tr[p].r, k - size(tr[p].l) - 1);
        }
    }

    int find_by_order(int k) {
```

```
        return find_by_order(root, k);
    }

    // 查询前驱：小于 x 的最大值
    int predecessor(int p, int x, int res = -1) {
        if (!p) return res;
        if (tr[p].val < x) {
            return predecessor(tr[p].r, x, tr[p].val);
        } else {
            return predecessor(tr[p].l, x, res);
        }
    }

    int predecessor(int x) {
        return predecessor(root, x);
    }

    // 查询后继：大于 x 的最小值
    int successor(int p, int x, int res = -1) {
        if (!p) return res;
        if (tr[p].val > x) {
            return successor(tr[p].l, x, tr[p].val);
        } else {
            return successor(tr[p].r, x, res);
        }
    }

    int successor(int x) {
        return successor(root, x);
    }
};

void slu() {
    AVL avl(1e5+1);
    int q; cin >> q;
    while (q--) {
        int op, x;
        cin >> op >> x;
        if (op == 1)      avl.insert(x);
        else if (op == 2) avl.erase(x);
        else if (op == 3) cout << avl.order_of_key(x)+1  << "\n";
        else if (op == 4) cout << avl.find_by_order(x-1) << "\n";
        else if (op == 5) cout << avl.predecessor(x)   << "\n";
        else              cout << avl.successor(x)     << "\n";
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    slu();
    return 0;
}
```

Tree\DSU

Tree\DSU\DSU.cpp

```cpp
struct DSU {
    std::vector<int> f, siz;

    DSU() {}
    DSU(int n) {
        init(n);
    }

    void init(int n) {
        f.resize(n);
        std::iota(f.begin(), f.end(), 0);
        siz.assign(n, 1);
    }

    int find(int x) {
        while (x != f[x]) {
            x = f[x] = f[f[x]];
        }
        return x;
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }

    bool merge(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) {
            return false;
        }
        siz[x] += siz[y];
        f[y] = x;
        return true;
    }

    int size(int x) {
        return siz[find(x)];
    }
};
/*
DSU dsu(n);
dsu.merge(a, b); // merge b into a (fa[b] = a)
dsu.same(a, b); // check if two nodes are in the same set
dsu.size(a); // get the size of the set of a
*/
```

Tree\FHQ_Treap

Tree\FHQ_Treap\FHQ_Treap.cpp

```cpp
#include "bits/stdc++.h"
using namespace std;

struct FHQ_Treap {
    struct Node {
        int val = 0;            // 节点存储的值
        int size = 0;           // 以该节点为根的子树大小（含重复节点）
        int l = 0, r = 0;       // 左右子节点下标（0 表示空）
        double priority = 0.0;  // 随机优先级（保持堆性质）
        Node() = default;
    };

    int root = 0;       // 根节点下标，0 表示空树
    int cnt = 0;        // 已使用节点数量
    vector<Node> node;  // 节点数组，0号节点为空节点（哨兵）

    // 构造函数，传入预分配大小
    FHQ_Treap(int size) {
        srand(time(0));  // 初始化随机数种子
        node.resize(size);
        root = 0;
        cnt = 0;
        node[0] = Node();  // 0号节点作为空节点
    }

    // 重置整个树，清空所有数据
    void clear() {
        for (int i = 0; i <= cnt; i++) node[i] = Node();
        root = 0;
        cnt = 0;
    }

    // 维护节点大小：等于左子树大小 + 右子树大小 + 自身计数
    void update(int x) {
        node[x].size = node[node[x].l].size + node[node[x].r].size + 1;
        ;
    }

    void split(int cur, int val, int &l, int &r) {  // 按照value分裂

        if (cur == 0) {  // 空树直接返回空
            l = r = 0;
            return;
        }

        // 左 :  小于等于val的元素
        // 右 :  大于val的元素
        if (node[cur].val <= val) {  // 当前节点应归到左子树
            l = cur;
            split(node[cur].r, val, node[cur].r, r);
            update(l);
        } else {  // 当前节点应归到右子树
            r = cur;
            split(node[cur].l, val, l, node[cur].l);
            update(r);
        }
    }
    void split1(int cur, int rank, int &l, int &r) {  // 按照rank分裂

        if (cur == 0) {  // 空树直接返回空
            l = r = 0;
            return;
        }

        // 左 :  小于等于rank的元素
        // 右 :  大于rank的元素
        if (node[node[cur].l].size + 1 <= rank) {  // 当前节点应归到左子树
            l = cur;
            split(node[cur].r, rank - node[node[cur].l].size - 1, node[cur].r, r);
            update(l);
        } else {  // 当前节点应归到右子树
            r = cur;
            split(node[cur].l, rank, l, node[cur].l);
            update(r);
        }
    }

    /*
     * 将两棵树 l 和 r 合并为一棵树，满足 BST 性质
     * 依据优先级保持堆性质
```

```
 */
int merge(int l, int r) {
    if (l == 0 || r == 0) return l + r;  // 一棵空树，返回另一棵
    if (node[l].priority >= node[r].priority) {
        node[l].r = merge(node[l].r, r);
        update(l);
        return l;
    } else {
        node[r].l = merge(l, node[r].l);
        update(r);
        return r;
    }
}

// 查找值为 val 的节点下标，找不到返回0
int find(int cur, int val) {
    if (cur == 0) return 0;
    if (node[cur].val == val) {
        return cur;
    } else if (node[cur].val > val) {
        return find(node[cur].l, val);
    } else {
        return find(node[cur].r, val);
    }
}

// 插入值 val
void insert(int val) {
    int l, r;
    split(root, val, l, r);  // 分裂成 <= val 和 > val 两棵树

    node[++cnt].val = val;
    node[cnt].size = 1;
    node[cnt].priority = (double)rand() / RAND_MAX;
    node[cnt].l = node[cnt].r = 0;

    root = merge(merge(l, cnt), r);
}

// 删除值 val
void erase(int val) {
    int l, r, lm;
    split(root, val, l, r);    // l包含 <= val, r > val
    split(l, val - 1, l, lm);  // lm只包含 val
    // lm 根节点即为一个 val 节点
    // 删除 lm 根节点，合并左右子树
    lm = merge(node[lm].l, node[lm].r);
    // 合并回去
    root = merge(merge(l, lm), r);
}

// 计算子树 cur 中小于 val 的元素个数（含计数）
int cnt_less_than(int val) {
    int l, r;
    split(root, val - 1, l, r);  // l 是 <= val-1, r 是 >= val
    int res = node[l].size;
    root = merge(l, r);  // 记得恢复原树结构
    return res;
}

// 返回 val 的排名（小于 val 的元素个数 + 1）
int order_of_val(int val) { return cnt_less_than(val) + 1; }

// 返回排名为 k 的元素（k 从1开始），找不到返回 -1
int find_by_order(int k) {
    int cur = root;
    while (cur) {
        int l_size = node[node[cur].l].size;  // 左子树大小

        if (k <= l_size) {            // k 在左子树中
            cur = node[cur].l;        // 转向左子树
        } else if (k > l_size + 1) {  // k 在右子树中
            k -= l_size + 1;
            cur = node[cur].r;
        } else {
            return node[cur].val;
        }
    }
    return -1;
}

// 找到子树 cur 最大值
int get_max(int cur) {
    if (cur == 0) return INT_MIN;
```

```
        while (node[cur].r) cur = node[cur].r;
        return node[cur].val;
    }

    // 找到子树 cur 最小值
    int get_min(int cur) {
        if (cur == 0) return INT_MAX;
        while (node[cur].l) cur = node[cur].l;
        return node[cur].val;
    }

    // 返回 val 的前驱（小于 val 的最大元素）
    int predecessor(int val) {
        int l, r;
        split(root, val - 1, l, r);  // l ≤ val-1, r ≥ val
        int res = get_max(l);
        root = merge(l, r);
        return res;
    }

    // 返回 val 的后继（大于 val 的最小元素）
    int successor(int val) {
        int l, r;
        split(root, val, l, r);  // l ≤ val, r > val
        int res = get_min(r);
        root = merge(l, r);
        return res;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    FHQ_Treap treap(1e5 + 7);
    int n;
    cin >> n;
    while (n--) {
        int op, x;
        cin >> op >> x;
        if (op == 1)
            treap.insert(x);  // 插入 x
        else if (op == 2)
            treap.erase(x);  // 删除 x
        else if (op == 3)
            cout << treap.order_of_val(x) << "\n";  // x 的排名
        else if (op == 4)
            cout << treap.find_by_order(x) << "\n";  // 找排名为 x 的元素
        else if (op == 5)
            cout << treap.predecessor(x) << "\n";  // x 的前驱
        else
            cout << treap.successor(x) << "\n";  // x 的后继
    }
    return 0;
}
```

Tree\FHQ_Treap\FHQ_Treap_可持久化.cpp

```cpp
// 可持久化平衡树，FHQ-Treap实现，不用词频压缩，C++版
// 认为一开始是0版本的树，为空树，实现如下操作，操作一共发生n次
// v 1 x : 基于v版本的树，增加一个x，生成新版本的树
// v 2 x : 基于v版本的树，删除一个x，生成新版本的树
// v 3 x : 基于v版本的树，查询x的排名，生成新版本的树状况=v版本状况
// v 4 x : 基于v版本的树，查询数据中排名为x的数，生成新版本的树状况=v版本状况
// v 5 x : 基于v版本的树，查询x的前驱，生成新版本的树状况=v版本状况
// v 6 x : 基于v版本的树，查询x的后继，生成新版本的树状况=v版本状况
// 不管什么操作，都基于某个v版本，操作完成后得到新版本的树，但v版本不会变化
// 如果x的前驱不存在，返回-2^31 + 1，如果x的后继不存在，返回+2^31 - 1
// 1 <= n <= 5 * 10^5
// -10^9 <= x <= +10^9
// 测试链接 : https://www.luogu.com.cn/problem/P3835
// 如下实现是C++的版本，C++版本和java版本逻辑完全一样
// 提交如下代码，可以通过所有测试用例

#include "bits/stdc++.h"

using namespace std;

const int MAXN = 500001;
const int MAXM = MAXN * 50;

int cnt = 0;
int head[MAXN];
int key[MAXM];
int ls[MAXM];
int rs[MAXM];
int siz[MAXM];
double priority[MAXM];

int copy(int i) {
    ++cnt;
    key[cnt] = key[i];
    ls[cnt] = ls[i];
    rs[cnt] = rs[i];
    siz[cnt] = siz[i];
    priority[cnt] = priority[i];
    return cnt;
}

void up(int i) {
    siz[i] = siz[ls[i]] + siz[rs[i]] + 1;
}

void split(int l, int r, int i, int num) {
    if (i == 0) {
        rs[l] = ls[r] = 0;
    } else {
        i = copy(i);
        if (key[i] <= num) {
            rs[l] = i;
            split(i, r, rs[i], num);
        } else {
            ls[r] = i;
            split(l, i, ls[i], num);
        }
        up(i);
    }
}

int merge(int l, int r) {
    if (l == 0 || r == 0) {
        return l + r;
    }
    if (priority[l] >= priority[r]) {
        l = copy(l);
        rs[l] = merge(rs[l], r);
        up(l);
        return l;
    } else {
        r = copy(r);
        ls[r] = merge(l, ls[r]);
        up(r);
        return r;
    }
}

void add(int v, int i, int num) {
    split(0, 0, i, num);
```

```
    int l = rs[0];
    int r = ls[0];
    ls[0] = rs[0] = 0;
    ++cnt;
    key[cnt] = num;
    siz[cnt] = 1;
    priority[cnt] = (double)rand() / RAND_MAX;
    head[v] = merge(merge(l, cnt), r);
}

void remove(int v, int i, int num) {
    split(0, 0, i, num);
    int lm = rs[0];
    int r = ls[0];
    split(0, 0, lm, num - 1);
    int l = rs[0];
    int m = ls[0];
    ls[0] = rs[0] = 0;
    head[v] = merge(merge(l, merge(ls[m], rs[m])), r);
}

int small(int i, int num) {
    if (i == 0) {
        return 0;
    }
    if (key[i] >= num) {
        return small(ls[i], num);
    } else {
        return siz[ls[i]] + 1 + small(rs[i], num);
    }
}

int index(int i, int x) {
    if (siz[ls[i]] >= x) {
        return index(ls[i], x);
    } else if (siz[ls[i]] + 1 < x) {
        return index(rs[i], x - siz[ls[i]] - 1);
    } else {
        return key[i];
    }
}

int pre(int i, int num) {
    if (i == 0) {
        return INT_MIN + 1;
    }
    if (key[i] >= num) {
        return pre(ls[i], num);
    } else {
        return max(key[i], pre(rs[i], num));
    }
}

int post(int i, int num) {
    if (i == 0) {
        return INT_MAX;
    }
    if (key[i] <= num) {
        return post(rs[i], num);
    } else {
        return min(key[i], post(ls[i], num));
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    srand(time(0));
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        int version, op, x;
        cin >> version >> op >> x;
        if (op == 1) {
            add(i, head[version], x);
        } else if (op == 2) {
            remove(i, head[version], x);
        } else {
            head[i] = head[version];
            if (op == 3) {
                cout << small(head[i], x) + 1 << "\n";
            } else if (op == 4) {
                cout << index(head[i], x) << "\n";
            } else if (op == 5) {
```

```
                cout << pre(head[i], x) << "\n";
            } else {
                cout << post(head[i], x) << "\n";
            }
        }
    }
    return 0;
}
```

Tree\FHQ_Treap\FHQ_Treap_指针.cpp

```cpp
#include "bits/stdc++.h"
using namespace std;

struct FHQTreap {
    // 无旋Treap：满足BST性质(val)和堆性质(priority)
    struct Node {
        int val;
        int size = 0;
        int priority;  // 随机优先级
        Node *left, *right;
        Node(int val) : val(val), priority(rand()), left(NULL), right(NULL), size(1) {}
        Node(int val, int priority) : val(val), priority(priority), left(NULL), right(NULL), size(1) {}
    };

    bool cmp(int a, int b) { return a <= b; }  // 自定义比较函数，满足BST性质
    Node *root;

    FHQTreap() : root(NULL) { srand((unsigned)time(nullptr)); }

    // 更新节点size
    void update(Node *root) {
        if (!root) return;
        root->size = 1;
        if (root->left) root->size += root->left->size;
        if (root->right) root->size += root->right->size;
    }

    // 合并两棵树a,b为root，满足BST和堆性质
    void merge(Node *&root, Node *a, Node *b) {
        if (!a || !b) {  // 存在空树，直接返回
            root = a ? a : b;
        } else {
            if (a->priority > b->priority) {
                root = a;
                merge(a->right, a->right, b);
            } else {
                root = b;
                merge(b->left, a, b->left);
            }
        }

        update(root);
    }

    // 按val拆分root为a,b，a中所有val <= val，b中所有val > val
    void split(Node *root, Node *&a, Node *&b, int val) {
        if (!root) {
            a = b = NULL;
            return;
        }

        if (cmp(root->val, val)) {
            a = root;
            split(root->right, a->right, b, val);
        } else {
            b = root;
            split(root->left, a, b->left, val);
        }

        update(root);
    }

    // 插入val
    void insert(int val) {
        Node *a, *b;
        split(root, a, b, val);
        merge(a, a, new Node(val));
        merge(root, a, b);
    }

    // 删除val（只删除一个）
    void erase(int val) {
        Node *a, *b, *c;
        split(root, a, b, val);
        split(a, a, c, val - 1);
        if (c) {
            // 删除c根节点，合并其左右子树
            merge(a, a, c->left);
            merge(a, a, c->right);
            delete c;  // 防止内存泄漏
```

```
        }
        merge(root, a, b);
    }

    // 返回val的排名（比val小的元素个数+1）
    int order_of_key(int val) {
        Node *a, *b;
        split(root, a, b, val - 1);
        int res = (a ? a->size : 0) + 1;
        merge(root, a, b);
        return res;
    }

    // 查询第k小元素，k从1开始，越界返回-1
    int find_by_order(int k) { return kth_query(root, k); }

    int kth_query(Node *root, int k) {
        if (!root) return -1;
        int leftsize = root->left ? root->left->size : 0;
        if (k <= leftsize) return kth_query(root->left, k);  // 左子树中第k小
        if (k == leftsize + 1) return root->val;             // 找到第k小元素
        return kth_query(root->right, k - leftsize - 1);     // 右子树中第k-leftsize-1小
    }

    // 查找val是否存在
    bool find(int val) {
        Node *a, *b;
        split(root, a, b, val);
        bool res = (a && find_max(a) == val);
        merge(root, a, b);
        return res;
    }

    // val的前驱（小于val的最大元素），不存在返回-1
    int predecessor(int val) {
        Node *a, *b;
        split(root, a, b, val - 1);
        int res = find_max(a);
        merge(root, a, b);
        return res;
    }

    // val的后继（大于val的最小元素），不存在返回-1
    int successor(int val) {
        Node *a, *b;
        split(root, a, b, val);
        int res = find_min(b);
        merge(root, a, b);
        return res;
    }

    // 中序打印
    void print(Node *root) {
        if (!root) return;
        print(root->left);
        cout << root->val << " ";
        print(root->right);
    }

    int find_max(Node *root) {
        if (!root) return -1;
        while (root->right) root = root->right;
        return root->val;
    }

    int find_min(Node *root) {
        if (!root) return -1;
        while (root->left) root = root->left;
        return root->val;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    FHQTreap treap;
    int n;
    cin >> n;
    while (n--) {
        int op, x;
        cin >> op >> x;
        if (op == 1)
            treap.insert(x);
```

```
        else if (op == 2)
            treap.erase(x);
        else if (op == 3)
            cout << treap.order_of_key(x) << "\n";
        else if (op == 4)
            cout << treap.find_by_order(x) << "\n";
        else if (op == 5)
            cout << treap.predecessor(x) << "\n";
        else if (op == 6)
            cout << treap.successor(x) << "\n";
    }

    return 0;
}
```

Tree\FHQ_Treap\FHQ_Treap_词频压缩.cpp

```cpp
#include "bits/stdc++.h"
using namespace std;

struct FHQ_Treap_词频压缩 {
    struct Node {
        int val = 0;            // 节点存储的值
        int cnt = 0;            // 当前值的出现次数（多重集支持）
        int size = 0;           // 以该节点为根的子树大小（含重复节点）
        int l = 0, r = 0;       // 左右子节点下标（0 表示空）
        double priority = 0.0;  // 随机优先级（保持堆性质）
        Node() = default;
    };

    int root = 0;       // 根节点下标，0 表示空树
    int cnt = 0;        // 已使用节点数量
    vector<Node> node;  // 节点数组，0号节点为空节点（哨兵）

    // 构造函数，传入预分配大小
    FHQ_Treap_词频压缩(int size) {
        srand(time(0));  // 初始化随机数种子
        node.resize(size);
        root = 0;
        cnt = 0;
        node[0] = Node();  // 0号节点作为空节点
    }

    // 重置整个树，清空所有数据
    void clear() {
        for (int i = 0; i <= cnt; i++) node[i] = Node();
        root = 0;
        cnt = 0;
    }

    // 维护节点大小：等于左子树大小 + 右子树大小 + 自身计数
    void update(int x) {
        node[x].size = node[node[x].l].size + node[node[x].r].size + node[x].cnt;
        ;
    }

    void split(int cur, int val, int &l, int &r) {  // 按照value分裂

        if (cur == 0) {  // 空树直接返回空
            l = r = 0;
            return;
        }

        // 左 :  小于等于val的元素
        // 右 :  大于val的元素
        if (node[cur].val <= val) {  // 当前节点应归到左子树
            l = cur;
            split(node[cur].r, val, node[cur].r, r);
            update(l);
        } else {  // 当前节点应归到右子树
            r = cur;
            split(node[cur].l, val, l, node[cur].l);
            update(r);
        }
    }
    void split1(int cur, int rank, int &l, int &r) {  // 按照rank分裂

        if (cur == 0) {  // 空树直接返回空
            l = r = 0;
            return;
        }

        // 左 :  小于等于rank的元素
        // 右 :  大于rank的元素
        if (node[node[cur].l].size + node[cur].cnt <= rank) {  // 当前节点应归到左子树
            l = cur;
            split(node[cur].r, rank - node[node[cur].l].size - node[cur].cnt, node[cur].r, r);
            update(l);
        } else {  // 当前节点应归到右子树
            r = cur;
            split(node[cur].l, rank, l, node[cur].l);
            update(r);
        }
    }

    /*
     * 将两棵树 l 和 r 合并为一棵树，满足 BST 性质
```

```
 * 依据优先级保持堆性质
 */
int merge(int l, int r) {
    if (l == 0 || r == 0) return l + r;  // 一棵空树，返回另一棵
    if (node[l].priority >= node[r].priority) {
        node[l].r = merge(node[l].r, r);
        update(l);
        return l;
    } else {
        node[r].l = merge(l, node[r].l);
        update(r);
        return r;
    }
}

// 查找值为 val 的节点下标，找不到返回0
int find(int cur, int val) {
    if (cur == 0) return 0;
    if (node[cur].val == val) {
        return cur;
    } else if (node[cur].val > val) {
        return find(node[cur].l, val);
    } else {
        return find(node[cur].r, val);
    }
}

// 修改节点 val 的计数，delta 可以为正（增加）或负（减少）
void change_cnt(int cur, int val, int delta) {
    if (node[cur].val == val) {
        node[cur].cnt += delta;
    } else if (node[cur].val > val) {
        change_cnt(node[cur].l, val, delta);
    } else {
        change_cnt(node[cur].r, val, delta);
    }
    update(cur);
}

// 插入值 val
void insert(int val) {
    if (find(root, val)) {
        // 如果存在该值，计数加1
        change_cnt(root, val, 1);
    } else {
        // 不存在该值，新建节点并合并
        int l, r;
        split(root, val, l, r);  // 分裂成 <= val 和 > val 两棵树
        node[++cnt].val = val;
        node[cnt].cnt = node[cnt].size = 1;
        node[cnt].priority = (double)rand() / RAND_MAX;
        node[cnt].l = node[cnt].r = 0;
        root = merge(merge(l, cnt), r);
    }
}

// 删除值 val
void erase(int val) {
    int cur = find(root, val);
    if (cur == 0) return;  // 不存在，直接返回
    if (node[cur].cnt > 1) {
        // 计数大于1，计数减1
        change_cnt(root, val, -1);
    } else {
        // 计数为1，物理删除节点，拆分合并实现
        int l, r, lm;
        split(root, val, l, r);    // l包含 <= val, r > val
        split(l, val - 1, l, lm);  // lm只包含 val
        root = merge(l, r);        // 删除lm对应节点
    }
}

// 计算子树 cur 中小于 val 的元素个数（含计数）
int cnt_less_than(int cur, int val) {
    if (cur == 0) return 0;
    if (node[cur].val >= val)
        return cnt_less_than(node[cur].l, val);
    else
        return node[node[cur].l].size + node[cur].cnt + cnt_less_than(node[cur].r, val);
}

// 返回 val 的排名（小于 val 的元素个数 + 1）
int order_of_val(int val) { return cnt_less_than(root, val) + 1; }
```

```
    // 返回排名为 k 的元素（k 从1开始），找不到返回 -1
    int find_by_order(int k) {
        int cur = root;
        while (cur) {
            int l_size = node[node[cur].l].size;// 左子树大小
            if (k <= l_size) {// k 在左子树中
                cur = node[cur].l;// 转向左子树
            } else if (k > l_size + node[cur].cnt) {// k 在右子树中
                k -= l_size + node[cur].cnt;
                cur = node[cur].r;
            } else {
                return node[cur].val;
            }
        }
        return -1;
    }

    // 找到子树 cur 最大值
    int get_max(int cur) {
        if (cur == 0) return INT_MIN;
        while (node[cur].r) cur = node[cur].r;
        return node[cur].val;
    }

    // 找到子树 cur 最小值
    int get_min(int cur) {
        if (cur == 0) return INT_MAX;
        while (node[cur].l) cur = node[cur].l;
        return node[cur].val;
    }

    // 返回 val 的前驱（小于 val 的最大元素）
    int predecessor(int val) {
        int l, r;
        split(root, val - 1, l, r);  // l ≤ val-1, r ≥ val
        int res = get_max(l);
        root = merge(l, r);
        return res;
    }

    // 返回 val 的后继（大于 val 的最小元素）
    int successor(int val) {
        int l, r;
        split(root, val, l, r);  // l ≤ val, r > val
        int res = get_min(r);
        root = merge(l, r);
        return res;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);


    FHQ_Treap_词频压缩 treap(1e5 + 7);
    int n;
    cin >> n;
    while (n--) {
        int op, x;
        cin >> op >> x;
        if (op == 1)
            treap.insert(x);  // 插入 x
        else if (op == 2)
            treap.erase(x);  // 删除 x
        else if (op == 3)
            cout << treap.order_of_val(x) << "\n";  // x 的排名
        else if (op == 4)
            cout << treap.find_by_order(x) << "\n";  // 找排名为 x 的元素
        else if (op == 5)
            cout << treap.predecessor(x) << "\n";  // x 的前驱
        else
            cout << treap.successor(x) << "\n";  // x 的后继
    }
    return 0;
}
```

Tree\Fenwick

Tree\Fenwick\Fenwick.cpp

```cpp
template <typename T>
struct Fenwick {
    int n;
    std::vector<T> a;

    Fenwick(int n_ = 0) {
        init(n_);
    }

    void init(int n_) {
        n = n_;
        a.assign(n, T{});
    }

    void add(int x, const T &v) {
        for (int i = x + 1; i <= n; i += i & -i) {
            // i += lowbit(i) -> 往上更新
            a[i - 1] = a[i - 1] + v;
        }
    }

    T sum(int x) {
        T ans{};
        for (int i = x; i > 0; i -= i & -i) {
            // i -= lowbit(i) -> 往下求和
            ans = ans + a[i - 1];
        }
        return ans;
    }

    T rangeSum(int l, int r) {
        return sum(r) - sum(l);
    }

    int select(const T &k) {
        int x = 0;
        T cur{};
        for (int i = 1 << std::__lg(n); i; i /= 2) {
            if (x + i <= n && cur + a[x + i - 1] <= k) {
                x += i;
                cur = cur + a[x - 1];
            }
        }
        return x;
    }
};
/*
https://www.luogu.com.cn/problem/P3368
add(x,v): 在x位置加上v
sum(x): 求[0,x)的和 // 左闭右开
rangeSum(l,r): 求[l,r)的和
select(k): 二分找到第一个大于等于k的位置 ? 存疑
*/
```

Tree\SegmentTree

Tree\SegmentTree\LazySegmentTree.cpp

```cpp
template<class Info, class Tag>
struct LazySegmentTree {

    int n;
    std::vector<Info> info;
    std::vector<Tag> tag;

    LazySegmentTree() : n(0) {}

    LazySegmentTree(int n_, Info v_ = Info()) {
        init(n_, v_);
    }

    template<class T>
    LazySegmentTree(std::vector<T> init_) {
        init(init_);
    }

    void init(int n_, Info v_ = Info()) {
        init(std::vector(n_, v_));
    }

    template<class T>
    void init(std::vector<T> init_) {
        n = init_.size();
        info.assign(4 << std::__lg(n), Info());
        tag.assign(4 << std::__lg(n), Tag());
        std::function<void(int, int, int)> build = [&](int p, int l, int r) {
            if (r - l == 1) {
                info[p] = init_[l];
                return;
            }
            int m = (l + r) / 2;
            build(2 * p, l, m);
            build(2 * p + 1, m, r);
            pull(p);
        };
        build(1, 0, n);
    }

    void pull(int p) {
        info[p] = info[2 * p] + info[2 * p + 1];
    }

    void apply(int p, const Tag &v) {
        info[p].apply(v);
        tag[p].apply(v);
    }

    void push(int p) {
        apply(2 * p, tag[p]);
        apply(2 * p + 1, tag[p]);
        tag[p] = Tag();
    }

    void modify(int p, int l, int r, int x, const Info &v) {
        if (r - l == 1) {
            info[p] = v;
            return;
        }
        int m = (l + r) / 2;
        push(p);
        if (x < m) {
            modify(2 * p, l, m, x, v);
        } else {
            modify(2 * p + 1, m, r, x, v);
        }
        pull(p);
    }

    void modify(int p, const Info &v) {
        modify(1, 0, n, p, v);
    }

    Info rangeQuery(int p, int l, int r, int x, int y) {
        if (l >= y || r <= x) {
            return Info();
        }
        if (l >= x && r <= y) {
            return info[p];
```

```
        }
        int m = (l + r) / 2;
        push(p);
        return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m, r, x, y);
    }

    Info rangeQuery(int l, int r) {
        return rangeQuery(1, 0, n, l, r);
    }

    void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
        if (l >= y || r <= x) {
            return;
        }
        if (l >= x && r <= y) {
            apply(p, v);
            return;
        }
        int m = (l + r) / 2;
        push(p);
        rangeApply(2 * p, l, m, x, y, v);
        rangeApply(2 * p + 1, m, r, x, y, v);
        pull(p);
    }

    void rangeApply(int l, int r, const Tag &v) {
        return rangeApply(1, 0, n, l, r, v);
    }

    void half(int p, int l, int r) {
        if (info[p].act == 0) {
            return;
        }
        if ((info[p].min + 1) / 2 == (info[p].max + 1) / 2) {
            apply(p, {-(info[p].min + 1) / 2});
            return;
        }
        int m = (l + r) / 2;
        push(p);
        half(2 * p, l, m);
        half(2 * p + 1, m, r);
        pull(p);
    }

    void half() {
        half(1, 0, n);
    }

    template<class F>
    int findFirst(int p, int l, int r, int x, int y, F &&pred) {
        if (l >= y || r <= x) {
            return -1;
        }
        if (l >= x && r <= y) {
            if (!pred(info[p], l, r)) {
                return -1;
            }
            if (r - l == 1) {
                return l;
            }
        }

        int m = (l + r) / 2;
        push(p);
        int res = findFirst(2 * p, l, m, x, y, pred);
        if (res == -1) {
            res = findFirst(2 * p + 1, m, r, x, y, pred);
        }
        return res;
    }

    template<class F>
    int findFirst(int l, int r, F &&pred) {
        return findFirst(1, 0, n, l, r, pred);
    }

    template<class F>
    int findLast(int p, int l, int r, int x, int y, F &&pred) {
        if (l >= y || r <= x) {
            return -1;
        }
        if (l >= x && r <= y) {
            if (!pred(info[p], l, r)) {
                return -1;
```

```
            }
            if (r - l == 1) {
                return l;
            }
        }

        int m = (l + r) / 2;
        push(p);
        int res = findLast(2 * p + 1, m, r, x, y, pred);
        if (res == -1) {
            res = findLast(2 * p, l, m, x, y, pred);
        }
        return res;
    }

    template<class F>
    int findLast(int l, int r, F &&pred) {
        return findLast(1, 0, n, l, r, pred);
    }

    void maintainL(int p, int l, int r, int pre) {
        if (info[p].difl > 0 && info[p].maxlowl < pre) {
            return;
        }
        if (r - l == 1) {
            info[p].max = info[p].maxlowl;
            info[p].maxl = info[p].maxr = l;
            info[p].maxlowl = info[p].maxlowr = -inf;
            return;
        }
        int m = (l + r) / 2;
        push(p);
        maintainL(2 * p, l, m, pre);
        pre = std::max(pre, info[2 * p].max);
        maintainL(2 * p + 1, m, r, pre);
        pull(p);
    }

    void maintainL() {
        maintainL(1, 0, n, -1);
    }

    void maintainR(int p, int l, int r, int suf) {
        if (info[p].difr > 0 && info[p].maxlowr < suf) {
            return;
        }
        if (r - l == 1) {
            info[p].max = info[p].maxlowl;
            info[p].maxl = info[p].maxr = l;
            info[p].maxlowl = info[p].maxlowr = -inf;
            return;
        }
        int m = (l + r) / 2;
        push(p);
        maintainR(2 * p + 1, m, r, suf);
        suf = std::max(suf, info[2 * p + 1].max);
        maintainR(2 * p, l, m, suf);
        pull(p);
    }

    void maintainR() {
        maintainR(1, 0, n, -1);
    }
};

struct Tag {
    bool act = false;
    int x = 0;

    void apply(const Tag &t) &{
        if (t.act) {
            act = true;
            x += t.x;
        }
    }
};

struct Info {
    bool act = false;
    int x = 0;

    void apply(const Tag &t) &{
        if (t.act) {
            x += t.x;
```

```
        }
    }
};

Info operator+(const Info &a, const Info &b) {
    if (!a.act && !b.act) return (Info){};
    if (!a.act) return b;
    if (!b.act) return a;

    Info res = {true};

    return res;
}
/*
// 查找 [l, r) 内第一个/ 最后一个 满足 pred(info) 的位置，不存在返回 -1
seg.findFirst(l, r, [&](const Info &info , int l, int r) { return info.sum >= k; });
seg.findLast(l, r, [&](const Info &info) { return info.x > 0; });
优先递归左子树，左边不行才去右边。
优先递归右子树，右边不行才去左边。
pred -> 先看区间左/右 子树如果满足条件就进入，否则扩大范围向右/左子树移动

ex:
[&](const Info &info, int _l, int _r) { find_first
    int _min = std::min(min , info.min); // 用外部变量维护当前 [l , _r) 最小值
    if (_min - (_r - L - 1) <= 0) { // 左子树满足要求进入
        return true;
    } else { // 扩大范围延伸右子树
        min = _min;
        return false;
    }
}

seg.modify(p, v);           // 单点修改：将位置 p 的信息改为 v (Info)
seg.rangeApply(l, r, v);    // 区间修改：[l, r) 范围应用标记 v (Tag)
seg.rangeQuery(l, r);       // 区间查询：查询 [l, r) 的聚合信息 (Info)

LazySegmentTree<Info, Tag> seg(n);
LazySegmentTree<Info, Tag> seg(_seg);
*/
```

Tree\SegmentTree\LazySegmentTree_rangeModify.cpp

```cpp
#include "bits/stdc++.h"
/*
关注b站<Feijoa_Li>谢谢喵
UID:248614274
https://space.bilibili.com/248614274
 */
using namespace std;

using u64 = unsigned long long;
using u32 = unsigned;
using u128 = unsigned __int128;
using i128 = __int128;

#define int long long
#define itn long long
#define Ist first
#define Ind second
#define endl "\n"
#define PII pair<int, int>
#define dbg cout << "line : " << __LINE__ << " "
constexpr int inf = 1e18;

template <class Info, class Tag>
struct LazySegmentTree {
    int n;
    std::vector<Info> info;
    std::vector<Tag> tag;

    LazySegmentTree() : n(0) {}

    LazySegmentTree(int n_, Info v_ = Info()) { init(n_, v_); }

    template <class T>
    LazySegmentTree(std::vector<T> init_) {
        init(init_);
    }

    void init(int n_, Info v_ = Info()) { init(std::vector(n_, v_)); }

    template <class T>
    void init(std::vector<T> init_) {
        n = init_.size();
        info.assign(4 << std::__lg(n), Info());
        tag.assign(4 << std::__lg(n), Tag());
        std::function<void(int, int, int)> build = [&](int p, int l, int r) {
            if (r - l == 1) {
                info[p] = init_[l];
                return;
            }
            int m = (l + r) / 2;
            build(2 * p, l, m);
            build(2 * p + 1, m, r);
            pull(p);
        };
        build(1, 0, n);
    }

    void pull(int p) { info[p] = info[2 * p] + info[2 * p + 1]; }

    void apply(int p, const Tag &v) {
        info[p].apply(v);
        tag[p].apply(v);
    }

    void push(int p) {
        apply(2 * p, tag[p]);
        apply(2 * p + 1, tag[p]);
        tag[p] = Tag();
    }

    void modify(int p, int l, int r, int x, const Info &v) {
        if (r - l == 1) {
            info[p] = v;
            return;
        }
        int m = (l + r) / 2;
        push(p);
        if (x < m) {
            modify(2 * p, l, m, x, v);
        } else {
```

```
        modify(2 * p + 1, m, r, x, v);
    }
    pull(p);
}

void modify(int p, const Info &v) { modify(1, 0, n, p, v); }

Info rangeQuery(int p, int l, int r, int x, int y) {
    if (l >= y || r <= x) {
        return Info();
    }
    if (l >= x && r <= y) {
        return info[p];
    }
    int m = (l + r) / 2;
    push(p);
    return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m, r, x, y);
}

Info rangeQuery(int l, int r) { return rangeQuery(1, 0, n, l, r); }

void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
    if (l >= y || r <= x) {
        return;
    }
    if (l >= x && r <= y) {
        apply(p, v);
        return;
    }
    int m = (l + r) / 2;
    push(p);
    rangeApply(2 * p, l, m, x, y, v);
    rangeApply(2 * p + 1, m, r, x, y, v);
    pull(p);
}

void rangeApply(int l, int r, const Tag &v) { return rangeApply(1, 0, n, l, r, v); }


void rangeModify(int p, int l, int r, int x, int y, int X) {
    if (l >= y || r <= x) return;  // 完全不在修改范围内

    // 以下是修改操作
    if (info[p].Max < X || info[p].Max == 0) return; // 该节点所有值都无需修改，直接返回
    if (r - l == 1) {
        info[p].Max = info[p].Sum = info[p].Max % X;
        return;
    }
    // end

    int m = (l + r) / 2;
    rangeModify(2 * p, l, m, x, y, X);
    rangeModify(2 * p + 1, m, r, x, y, X);
    pull(p);
}

void rangeModify(int l, int r, int X) { rangeModify(1, 0, n, l, r, X); }

void half(int p, int l, int r) {
    if (info[p].act == 0) {
        return;
    }
    if ((info[p].min + 1) / 2 == (info[p].max + 1) / 2) {
        apply(p, {-(info[p].min + 1) / 2});
        return;
    }
    int m = (l + r) / 2;
    push(p);
    half(2 * p, l, m);
    half(2 * p + 1, m, r);
    pull(p);
}

void half() { half(1, 0, n); }

template <class F>
int findFirst(int p, int l, int r, int x, int y, F &&pred) {
    if (l >= y || r <= x) {
        return -1;
    }
    if (l >= x && r <= y && !pred(info[p])) {
        return -1;
    }
    if (r - l == 1) {
```

```
            return l;
        }
        int m = (l + r) / 2;
        push(p);
        int res = findFirst(2 * p, l, m, x, y, pred);
        if (res == -1) {
            res = findFirst(2 * p + 1, m, r, x, y, pred);
        }
        return res;
    }

    template <class F>
    int findFirst(int l, int r, F &&pred) {
        return findFirst(1, 0, n, l, r, pred);
    }

    template <class F>
    int findLast(int p, int l, int r, int x, int y, F &&pred) {
        if (l >= y || r <= x) {
            return -1;
        }
        if (l >= x && r <= y && !pred(info[p])) {
            return -1;
        }
        if (r - l == 1) {
            return l;
        }
        int m = (l + r) / 2;
        push(p);
        int res = findLast(2 * p + 1, m, r, x, y, pred);
        if (res == -1) {
            res = findLast(2 * p, l, m, x, y, pred);
        }
        return res;
    }

    template <class F>
    int findLast(int l, int r, F &&pred) {
        return findLast(1, 0, n, l, r, pred);
    }

    void maintainL(int p, int l, int r, int pre) {
        if (info[p].difl > 0 && info[p].maxlowl < pre) {
            return;
        }
        if (r - l == 1) {
            info[p].max = info[p].maxlowl;
            info[p].maxl = info[p].maxr = l;
            info[p].maxlowl = info[p].maxlowr = -inf;
            return;
        }
        int m = (l + r) / 2;
        push(p);
        maintainL(2 * p, l, m, pre);
        pre = std::max(pre, info[2 * p].max);
        maintainL(2 * p + 1, m, r, pre);
        pull(p);
    }

    void maintainL() { maintainL(1, 0, n, -1); }

    void maintainR(int p, int l, int r, int suf) {
        if (info[p].difr > 0 && info[p].maxlowr < suf) {
            return;
        }
        if (r - l == 1) {
            info[p].max = info[p].maxlowl;
            info[p].maxl = info[p].maxr = l;
            info[p].maxlowl = info[p].maxlowr = -inf;
            return;
        }
        int m = (l + r) / 2;
        push(p);
        maintainR(2 * p + 1, m, r, suf);
        suf = std::max(suf, info[2 * p + 1].max);
        maintainR(2 * p, l, m, suf);
        pull(p);
    }

    void maintainR() { maintainR(1, 0, n, -1); }
};

struct Tag {
    void apply(const Tag &t) & {}
```

```
};

struct Info {
    bool act = false;
    int Max = -inf;
    int Sum = 0;

    void apply(const Tag &t) & {}
};

Info operator+(const Info &a, const Info &b) {
    if (!a.act) return b;
    if (!b.act) return a;
    return Info{true, max(a.Max, b.Max), a.Sum + b.Sum};
}
// LazySegmentTree

/*

*/
```

Tree\SegmentTree\SegmentTree.cpp

```cpp
template<class Info>
struct SegmentTree {
    int n;
    std::vector<Info> info;
    SegmentTree() : n(0) {}
    SegmentTree(int n_, Info v_ = Info()) {
        init(n_, v_);
    }
    template<class T>
    SegmentTree(std::vector<T> init_) {
        init(init_);
    }
    void init(int n_, Info v_ = Info()) {
        init(std::vector(n_, v_));
    }
    template<class T>
    void init(std::vector<T> init_) {
        n = init_.size();
        info.assign(4 << std::__lg(n), Info());
        std::function<void(int, int, int)> build = [&](int p, int l, int r) {
            if (r - l == 1) {
                info[p] = init_[l];
                return;
            }
            int m = (l + r) / 2;
            build(2 * p, l, m);
            build(2 * p + 1, m, r);
            pull(p);
        };
        build(1, 0, n);
    }
    void pull(int p) {
        info[p] = info[2 * p] + info[2 * p + 1];
    }
    void modify(int p, int l, int r, int x, const Info &v) {
        if (r - l == 1) {
            info[p] = v;
            return;
        }
        int m = (l + r) / 2;
        if (x < m) {
            modify(2 * p, l, m, x, v);
        } else {
            modify(2 * p + 1, m, r, x, v);
        }
        pull(p);
    }
    void modify(int p, const Info &v) {
        modify(1, 0, n, p, v);
    }
    Info rangeQuery(int p, int l, int r, int x, int y) {
        if (l >= y || r <= x) {
            return Info();
        }
        if (l >= x && r <= y) {
            return info[p];
        }
        int m = (l + r) / 2;
        return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m, r, x, y);
    }
    Info rangeQuery(int l, int r) {
        return rangeQuery(1, 0, n, l, r);
    }
    template<class F>
    int findFirst(int p, int l, int r, int x, int y, F &&pred) {
        if (l >= y || r <= x) {
            return -1;
        }
        if (l >= x && r <= y && !pred(info[p])) {
            return -1;
        }
        if (r - l == 1) {
            return l;
        }
        int m = (l + r) / 2;
        int res = findFirst(2 * p, l, m, x, y, pred);
        if (res == -1) {
            res = findFirst(2 * p + 1, m, r, x, y, pred);
        }
        return res;
```

```
    }
    template<class F>
    int findFirst(int l, int r, F &&pred) {
        return findFirst(1, 0, n, l, r, pred);
    }
    template<class F>
    int findLast(int p, int l, int r, int x, int y, F &&pred) {
        if (l >= y || r <= x) {
            return -1;
        }
        if (l >= x && r <= y && !pred(info[p])) {
            return -1;
        }
        if (r - l == 1) {
            return l;
        }
        int m = (l + r) / 2;
        int res = findLast(2 * p + 1, m, r, x, y, pred);
        if (res == -1) {
            res = findLast(2 * p, l, m, x, y, pred);
        }
        return res;
    }
    template<class F>
    int findLast(int l, int r, F &&pred) {
        return findLast(1, 0, n, l, r, pred);
    }
};

struct Info {

};

Info operator+(Info l, const Info &r) {

}
```

Tree\SegmentTree\Li_Chao_seg_Tree

Tree\SegmentTree\Li_Chao_seg_Tree\LstTree.cpp

```cpp
template<class Info, class Tag>
struct LazySegmentTree {

    int n;
    std::vector<Info> info;
    std::vector<Tag> tag;

    LazySegmentTree() : n(0) {}

    LazySegmentTree(int n_, Info v_ = Info()) {
        init(n_, v_);
    }

    template<class T>
    LazySegmentTree(std::vector<T> init_) {
        init(init_);
    }

    void init(int n_, Info v_ = Info()) {
        init(std::vector(n_, v_));
    }

    template<class T>
    void init(std::vector<T> init_) {
        n = init_.size();
        info.assign(4 << std::__lg(n), Info());
        tag.assign(4 << std::__lg(n), Tag());
        std::function<void(int, int, int)> build = [&](int p, int l, int r) {
            if (r - l == 1) {
                info[p] = init_[l];
                return;
            }
            int m = (l + r) / 2;
            build(2 * p, l, m);
            build(2 * p + 1, m, r);
            pull(p);
        };
        build(1, 0, n);
    }

    void pull(int p) {
        info[p] = info[2 * p] + info[2 * p + 1];
    }

    void apply(int p, const Tag &v) {
        info[p].apply(v);
        tag[p].apply(v);
    }

    void push(int p) {
        apply(2 * p, tag[p]);
        apply(2 * p + 1, tag[p]);
        tag[p] = Tag();
    }

    void modify(int p, int l, int r, int x, const Info &v) {
        if (r - l == 1) {
            info[p] = v;
            return;
        }
        int m = (l + r) / 2;
        push(p);
        if (x < m) {
            modify(2 * p, l, m, x, v);
        } else {
            modify(2 * p + 1, m, r, x, v);
        }
        pull(p);
    }

    void modify(int p, const Info &v) {
        modify(1, 0, n, p, v);
    }

    Info rangeQuery(int p, int l, int r, int x, int y) {
        if (l >= y || r <= x) {
            return Info();
        }
        if (l >= x && r <= y) {
            return info[p];
```

```
        }
        int m = (l + r) / 2;
        push(p);
        return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m, r, x, y);
    }

    Info rangeQuery(int l, int r) {
        return rangeQuery(1, 0, n, l, r);
    }

    void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
        if (l >= y || r <= x) {
            return;
        }
        if (l >= x && r <= y) {
            apply(p, v);
            return;
        }
        int m = (l + r) / 2;
        push(p);
        rangeApply(2 * p, l, m, x, y, v);
        rangeApply(2 * p + 1, m, r, x, y, v);
        pull(p);
    }

    void rangeApply(int l, int r, const Tag &v) {
        return rangeApply(1, 0, n, l, r, v);
    }
};

struct Tag {
    bool act = false;
    int x = 0;

    void apply(const Tag &t) &{
        if (t.act) {
            act = true;
            x += t.x;
        }
    }
};

struct Info {
    bool act = false;
    int x = 0;

    void apply(const Tag &t) &{
        if (t.act) {
            x += t.x;
        }
    }
};

Info operator+(const Info &a, const Info &b) {
    if (!a.act) return b;
    if (!b.act) return a;

    Info res = {true};

    return res;
}
/*
seg.modify(p, v);           // 单点修改：将位置 p 的信息改为 v (Info)
seg.rangeApply(l, r, v);    // 区间修改：[l, r) 范围应用标记 v (Tag)
seg.rangeQuery(l, r);       // 区间查询：查询 [l, r) 的聚合信息 (Info)

LazySegmentTree<Info, Tag> seg(n);
LazySegmentTree<Info, Tag> seg(_seg);
*/
```

Tree\SegmentTree\Li_Chao_seg_Tree\q_Max.cpp

```cpp
#include <iostream>
#include <algorithm>
#include <cassert>

constexpr long long INF = 1'000'000'000'000'000'000;
constexpr long long NEG_INF = -INF;  // 最大值查询用负无穷

struct Line {
    int k;
    long long b;
    Line(int _k = 0, long long _b = NEG_INF) : k(_k), b(_b) {}
    long long value(int x) const {
        return 1LL * k * x + b;
    }
};

struct Node {
    Line line;
    Node *lc, *rc;
    Node(const Line &line) : line(line), lc(nullptr), rc(nullptr) {}
};

class LiChaoTree {
    Node *root = nullptr;
    int left, right; // 区间[left, right)

    void modify(Node *&p, int l, int r, const Line &line) {
        if (!p) {
            p = new Node(line);
            return;
        }
        int m = (l + r) / 2;
        bool leftBetter = p->line.value(l) > line.value(l);  // 【改为最大值判定】
        bool midBetter = p->line.value(m) > line.value(m);   // 【改为最大值判定】
        if (!midBetter)
            std::swap(p->line, const_cast<Line&>(line));
        if (r - l == 1)
            return;
        if (leftBetter != midBetter) {
            modify(p->lc, l, m, line);
        } else {
            modify(p->rc, m, r, line);
        }
    }

    long long query(Node *p, int l, int r, int x) const {
        if (!p) return NEG_INF;  // 【空节点返回负无穷】
        long long cur = p->line.value(x);
        if (r - l == 1) return cur;
        int m = (l + r) / 2;
        if (x < m) {
            return std::max(cur, query(p->lc, l, m, x));  // 【min改为max】
        } else {
            return std::max(cur, query(p->rc, m, r, x));  // 【min改为max】
        }
    }

    Node* merge(Node *p, Node *q, int l, int r) {
        if (!p) return q;
        if (!q) return p;
        int m = (l + r) / 2;
        p->lc = merge(p->lc, q->lc, l, m);
        p->rc = merge(p->rc, q->rc, m, r);
        modify(p, l, r, q->line);
        return p;
    }

    LiChaoTree(int l, int r) : root(nullptr), left(l), right(r) {}

    void insert(int k, long long b) {
        modify(root, left, right, Line(k, b));
    }

    long long query(int x) const {
        assert(x >= left && x < right);
        return query(root, left, right, x);
    }

    void merge(LiChaoTree &other) {
        root = merge(root, other.root, left, right);
```

```
        other.root = nullptr;
    }
};

int main() {
    LiChaoTree tree1(-100000, 100001);
    LiChaoTree tree2(-100000, 100001);

    // 插入线段 y=2x+3 和 y=-1x+10 到 tree1
    tree1.insert(2, 3);
    tree1.insert(-1, 10);

    // 插入线段 y=1x+0 和 y=0x+5 到 tree2
    tree2.insert(1, 0);
    tree2.insert(0, 5);

    std::cout << "tree1 max at x=5: " << tree1.query(5) << "\n"; // max(2*5+3=13, -1*5+10=5) = 13
    std::cout << "tree2 max at x=5: " << tree2.query(5) << "\n"; // max(1*5+0=5, 0+5=5) = 5

    // 合并tree2到tree1
    tree1.merge(tree2);

    std::cout << "after merge tree1 max at x=5: " << tree1.query(5) << "\n"; // max(13,5) = 13

    return 0;
}
```

Tree\SegmentTree\Li_Chao_seg_Tree\q_Min.cpp

```cpp
#include <iostream>
#include <algorithm>
#include <cassert>

constexpr long long INF = 1'000'000'000'000'000'000;

struct Line {
    int k;
    long long b;
    Line(int _k = 0, long long _b = INF) : k(_k), b(_b) {}
    long long value(int x) const {
        return 1LL * k * x + b;
    }
};

struct Node {
    Line line;
    Node *lc, *rc;
    Node(const Line &line) : line(line), lc(nullptr), rc(nullptr) {}
};

class LiChaoTree {
    Node *root = nullptr;
    int left, right; // 区间[left, right)

    void modify(Node *&p, int l, int r, const Line &line) {
        if (!p) {
            p = new Node(line);
            return;
        }
        int m = (l + r) / 2;
        bool leftBetter = p->line.value(l) < line.value(l);
        bool midBetter = p->line.value(m) < line.value(m);
        if (!midBetter)
            std::swap(p->line, const_cast<Line&>(line));
        if (r - l == 1)
            return;
        if (leftBetter != midBetter) {
            modify(p->lc, l, m, line);
        } else {
            modify(p->rc, m, r, line);
        }
    }

    long long query(Node *p, int l, int r, int x) const {
        if (!p) return INF;
        long long cur = p->line.value(x);
        if (r - l == 1) return cur;
        int m = (l + r) / 2;
        if (x < m) {
            return std::min(cur, query(p->lc, l, m, x));
        } else {
            return std::min(cur, query(p->rc, m, r, x));
        }
    }

    Node* merge(Node *p, Node *q, int l, int r) {
        if (!p) return q;
        if (!q) return p;
        int m = (l + r) / 2;
        p->lc = merge(p->lc, q->lc, l, m);
        p->rc = merge(p->rc, q->rc, m, r);
        modify(p, l, r, q->line);
        return p;
    }

    LiChaoTree(int l, int r) : root(nullptr), left(l), right(r) {}

    void insert(int k, long long b) {
        modify(root, left, right, Line(k, b));
    }

    long long query(int x) const {
        assert(x >= left && x < right);
        return query(root, left, right, x);
    }

    // 把other合并进来，合并后other无效
    void merge(LiChaoTree &other) {
        root = merge(root, other.root, left, right);
```

```
        other.root = nullptr; // 避免double free
    }
};

int main() {
    // 区间[-100000, 100001)
    LiChaoTree tree1(-100000, 100001);
    LiChaoTree tree2(-100000, 100001);

    // 向tree1插入 y = 2x + 3
    tree1.insert(2, 3);
    // 向tree1插入 y = -1x + 10
    tree1.insert(-1, 10);

    // 向tree2插入 y = 1x + 0
    tree2.insert(1, 0);
    // 向tree2插入 y = 0x + 5
    tree2.insert(0, 5);

    // 查询tree1在x=5处的最小值
    std::cout << "tree1 query x=5: " << tree1.query(5) << "\n"; // 计算 y = min(2*5+3, -1*5+10) = min(13,5) = 5

    // 查询tree2在x=5处的最小值
    std::cout << "tree2 query x=5: " << tree2.query(5) << "\n"; // min(5+0, 0+5) = 5

    // 合并tree2到tree1
    tree1.merge(tree2);

    // 合并后查询tree1 x=5最小值
    std::cout << "after merge tree1 query x=5: " << tree1.query(5) << "\n";

    // tree2已合并，不能再用
    return 0;
}
```

Tree\Trie

Tree\Trie\01Trie.cpp

```
#include "bits/stdc++.h"
/*
_______  ______       ___         ___      _______      _______
|\  _____\|\  ___ \     |\  \       |\  \    |\   __  \    |\   __  \
\ \  \__/ \ \   __/|    \ \  \      \ \  \   \ \  \|\  \   \ \  \|\  \
 \ \   __\ \ \  \_|/__   \ \  \   __ \ \  \   \ \  \\\  \   \ \   __  \
  \ \  \_|  \ \  \_|\ \   \ \  \ |\  \\\_\  \   \ \  \\\  \   \ \  \ \  \
   \ \__\    \ \_______\   \ \__\\ \_______\   \ \_______\   \ \__\ \__\
 \|__|     \|_______|    \|__| \|_______|    \|_______|    \|__|\|__|
 */
using namespace std;
#define int long long
#define endl "\n"
#define PII pair<int,int>
#define x first
#define y second

const int MAXN = 150001;
vector<vector<int> > tree(MAXN, vector<int>(2, 0));
vector<int> End(MAXN, 0), Pass(MAXN, 0);
int cnt = 1;

void Build() {
 cnt = 1;
}

void Insert(int num) {
 int cur = 1; //头节点
 Pass[cur]++;
 for (int i = 29; i >= 0; i--) {
  int path = (num >> i) & 1;
  if (!tree[cur][path]) {
   tree[cur][path] = ++cnt;
  }
  cur = tree[cur][path];
  Pass[cur]++;
 }
 End[cur]++;
}

int Search(int num) {
 int cur = 1;
 for (int i = 29; i >= 0; i--) {
  int path = (num >> i) & 1;
  if (!tree[cur][path]) {
   return 0;
  }
  cur = tree[cur][path];
 }
 return End[cur];
}

int PrefixNumber(int num) {
 //有多少个以num开头的数字
 int cur = 1;
 for (int i = 29; i >= 0; i--) {
  int path = (num >> i) & 1;
  if (!tree[cur][path]) {
   return 0;
  }
  cur = tree[cur][path];
 }
 return Pass[cur];
}

void Delete(int num) {
 //删除以num开头的数字
 if (Search(num) > 0) {
  int cur = 1;
  Pass[cur]--;
  for (int i = 29; i >= 0; i--) {
   int path = (num >> i) & 1;
   int id = tree[cur][path];
   if (--Pass[id] == 0) {
    tree[cur][path] = 0;
    return;
   }
   cur = tree[cur][path];
  }
  End[cur]--;
```

```
 }
}

void clear() {
 for (int i = 1; i <= cnt; i++) {
  for (int j = 0; j < 2; j++) {
   tree[i][j] = 0;
  }
  End[i] = 0;
  Pass[i] = 0;
 }
}

void slu() {
}

signed main() {
 ios_base::sync_with_stdio(0);
 cin.tie(0);
 cout.tie(0);
 int T;
 // cin >> T;
 T = 1;
 while (T--)slu();
}
```

Tree\Trie\Trie.cpp

```
#include "bits/stdc++.h"
/*
关注b站<Feijoa_Li>谢谢喵
UID:248614274
https://space.bilibili.com/248614274
_______  ______       ___          ___      ________      ________
|\  _____\|\  ___ \     |\  \       |\  \    |\   __  \    |\   __  \
\ \  \__/ \ \   __/|    \ \  \      \ \  \   \ \  \|\  \   \ \  \|\  \
 \ \   __\ \ \  \_|/__   \ \  \   __ \ \  \   \ \  \\\  \   \ \   __  \
  \ \  \_|  \ \  \_|\ \   \ \  \ |\  \\\_\  \   \ \  \\\  \   \ \  \ \  \
   \ \__\    \ \_______\   \ \__\\ \_______\   \ \_______\   \ \__\ \__\
    \|__|     \|_______|    \|__| \|_______|    \|_______|    \|__|\|__|
 */
using namespace std;

using u64 = unsigned long long;
using u32 = unsigned;
using u128 = unsigned __int128;
using i128 = __int128;

#define int long long
#define Ist first
#define Ind second
#define itn long long
#define O (int)0
#define endl "\n"
#define PII pair<int, int>
const int inf = 1e18;

const int MAXN = 150001;
vector<vector<int> > tree(MAXN, vector<int>(26, 0));
vector<int> End(MAXN, 0), Pass(MAXN, 0);
int cnt = 1;

void Build() { cnt = 1; }

void Insert(string word) {
    int cur = 1;  // 头节点
    Pass[cur]++;
    for (int i = 0, path; i < word.length(); i++) {
        path = word[i] - 'a';
        if (tree[cur][path] == 0) {
            // 新建节点
            tree[cur][path] = ++cnt;
        }
        cur = tree[cur][path];  // 跳过去
        Pass[cur]++;            // pass++
    }
    End[cur]++;
}

int Search(string word) {
    int cur = 1;
    for (int i = 0, path; i < word.length(); i++) {
        path = word[i] - 'a';
        if (tree[cur][path] == 0) {
            return 0;
        }
        cur = tree[cur][path];
    }
    return End[cur];
}

int PrefixNumber(string pre) {
    // 有多少个以pre开头的单词
    int cur = 1;
    for (int i = 0, path; i < pre.length(); i++) {
        path = pre[i] - 'a';
        if (tree[cur][path] == 0) {
            return 0;
        }
        cur = tree[cur][path];
    }
    return Pass[cur];
}

void Delete(string word) {
    if (Search(word) > 0) {
        int cur = 1;
        Pass[cur]--;
```

```
        for (int i = 0, path; i < word.length(); i++) {
            path = word[i] - 'a';
            int id = tree[cur][path];
            if (--Pass[id] == 0) {
                tree[cur][path] = 0;
                return;
            }
            cur = tree[cur][path];
        }
        End[cur]--;
    }
}

void clear() {
    for (int i = 1; i <= cnt; i++) {
        for (int j = 0; j < 26; j++) {
            tree[i][j] = 0;
        }
        End[i] = 0;
        Pass[i] = 0;
    }
}

void slu() {}

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    int T;
    // cin >> T;
    T = 1;
    while (T--) slu();
}
```

Tree\平衡树

Tree\平衡树\其他平衡树A.cpp

```cpp
#include "bits/stdc++.h"
/*
关注b站<Feijoa_Li>谢谢喵
UID:248614274
https://space.bilibili.com/248614274
 */
using namespace std;
using u64 = unsigned long long;
using u32 = unsigned;
using u128 = unsigned __int128;
using i128 = __int128;

#define int long long
#define Ist first
#define Ind second
#define itn long long
#define O (int)0
#define endl "\n"
#define PII pair<int,int>
const int inf = 1e18;

/**   其他平衡树
 *    2023-08-04: https://ac.nowcoder.com/acm/contest/view-submission?submissionId=63246177
**/
struct Node {
    Node *l = nullptr;
    Node *r = nullptr;
    int sum = 0;
    int sumodd = 0;

    Node(Node *t) {
        if (t) {
            *this = *t;
        }
    }
};

Node *add(Node *t, int l, int r, int x, int v) {
    t = new Node(t);
    t->sum += v;
    t->sumodd += (x % 2) * v;
    if (r - l == 1) {
        return t;
    }
    int m = (l + r) / 2;
    if (x < m) {
        t->l = add(t->l, l, m, x, v);
    } else {
        t->r = add(t->r, m, r, x, v);
    }
    return t;
}

int query1(Node *t1, Node *t2, int l, int r, int k) {
    if (r - l == 1) {
        return l;
    }
    int m = (l + r) / 2;
    int odd = (t1 && t1->r ? t1->r->sumodd : 0) - (t2 && t2->r ? t2->r->sumodd : 0);
    int cnt = (t1 && t1->r ? t1->r->sum : 0) - (t2 && t2->r ? t2->r->sum : 0);
    if (odd > 0 || cnt > k) {
        return query1(t1 ? t1->r : t1, t2 ? t2->r : t2, m, r, k);
    } else {
        return query1(t1 ? t1->l : t1, t2 ? t2->l : t2, l, m, k - cnt);
    }
}

std::array<int, 3> query2(Node *t1, Node *t2, int l, int r, int k) {
    if (r - l == 1) {
        int cnt = (t1 ? t1->sumodd : 0) - (t2 ? t2->sumodd : 0);
        return {l, cnt, k};
    }
    int m = (l + r) / 2;
    int cnt = (t1 && t1->r ? t1->r->sumodd : 0) - (t2 && t2->r ? t2->r->sumodd : 0);
    if (cnt > k) {
        return query2(t1 ? t1->r : t1, t2 ? t2->r : t2, m, r, k);
    } else {
        return query2(t1 ? t1->l : t1, t2 ? t2->l : t2, l, m, k - cnt);
    }
}
```

```
/*

*/
void slu() {
}

signed main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    int t = 1;
    cin >> t;

    while (t--)slu();
    return 0;
}
```

Tree\平衡树\可持久化的权重平衡树（Weight-Balanced Leaf Tree, WBLT）.cpp

```cpp
#include <bits/stdc++.h>

using i64 = long long;
using u64 = unsigned long long;
using u32 = unsigned;

using u128 = unsigned __int128;
using i128 = __int128;

constexpr i64 inf = 1E18;

constexpr double ALPHA = 0.292;

struct Node {
    union {
        Node *l;
        int val;
    };
    Node *r;
    i128 siz;

    void pull() {
        siz = l->siz + r->siz;
    }
};

bool tooHeavy(i128 a, i128 b) {
    return b < ALPHA * (a + b);
}

std::pair<Node *, Node *> cut(Node *x) {
    return {x->l, x->r};
}

Node *join(Node *x, Node *y) {
    Node *t = new Node {.l = x, .r = y};
    t->pull();
    return t;
}

Node *merge(Node *x, Node *y) {
    if (!x) {
        return y;
    }
    if (!y) {
        return x;
    }
    if (tooHeavy(x->siz, y->siz)) {
        auto [a, b] = cut(x);
        if (tooHeavy(b->siz + y->siz, a->siz)) {
            auto [c, d] = cut(b);
            return merge(merge(a, c), merge(d, y));
        } else {
            return merge(a, merge(b, y));
        }
    } else if (tooHeavy(y->siz, x->siz)) {
        auto [a, b] = cut(y);
        if (tooHeavy(x->siz + a->siz, b->siz)) {
            auto [c, d] = cut(a);
            return merge(merge(x, c), merge(d, b));
        } else {
            return merge(merge(x, a), b);
        }
    } else {
        return join(x, y);
    }
}

int query(Node *t, i64 x) {
    if (!t->r) {
        return t->val;
    }
    if (x < t->l->siz) {
        return query(t->l, x);
    } else {
        return query(t->r, x - t->l->siz);
    }
}

int main() {
```

```
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int Q;
    std::cin >> Q;

    std::vector<Node *> t(Q + 2);
    t[0] = new Node {.val = 0, .siz = 1};
    t[1] = new Node {.val = 1, .siz = 1};

    for (int i = 1; i <= Q; i++) {
        int L, R;
        i64 X;
        std::cin >> L >> R >> X;

        if (t[L]->siz < inf) {
            t[i + 1] = merge(t[L], t[R]);
        } else {
            t[i + 1] = t[L];
        }

        X--;
        std::cout << query(t[i + 1], X) << "\n";
    }

    return 0;
}
```

Tree\树状数组（Fenwick 新版）

Tree\树状数组（Fenwick 新版）\树状数组（Fenwick 新版）.cpp

```cpp
template<typename T>
struct Fenwick {
    int n;
    std::vector<T> a;

    Fenwick(int n_ = 0) {
        init(n_);
    }

    void init(int n_) {
        n = n_;
        a.assign(n, T{});
    }

    void add(int x, const T &v) {
        for (int i = x + 1; i <= n; i += i & -i) {
            a[i - 1] = a[i - 1] + v;
        }
    }

    T sum(int x) {
        T ans{};
        for (int i = x; i > 0; i -= i & -i) {
            ans = ans + a[i - 1];
        }
        return ans;
    }

    T rangeSum(int l, int r) {
        return sum(r) - sum(l);
    }

    int select(const T &k) {
        int x = 0;
        T cur{};
        for (int i = 1 << std::__lg(n); i; i /= 2) {
            if (x + i <= n && cur + a[x + i - 1] <= k) {
                x += i;
                cur = cur + a[x - 1];
            }
        }
        return x;
    }
};
```

Tree\笛卡尔树

Tree\笛卡尔树\zcy.cpp

```cpp
    // 笛卡尔树模版(C++版)
// 给定一个长度为n的数组arr，下标从1开始
// 构建一棵二叉树，下标按照搜索二叉树组织，值按照小根堆组织
// 建树的过程要求时间复杂度O(n)
// 建树之后，为了验证
// 打印，i * (left[i] + 1)，所有信息异或起来的值
// 打印，i * (right[i] + 1)，所有信息异或起来的值
// 1 <= n <= 10^7
// 测试链接 : https://www.luogu.com.cn/problem/P5854
// 如下实现是C++的版本，C++版本和java版本逻辑完全一样
// 提交如下代码，可以通过所有测试用例

#include <iostream>
#include <vector>
#include <stack>
#include <cstdio>

#define LL long long

using namespace std;

const int MAXN = 10000001;

int arr[MAXN];
int ls[MAXN];
int rs[MAXN];
int sta[MAXN];
int n;

void build() {
    int top = 0;
    for (int i = 1; i <= n; i++) {
        int pos = top;
        while (pos > 0 && arr[sta[pos]] > arr[i]) {
            pos--;
        }
        if (pos > 0) {
            rs[sta[pos]] = i;
        }
        if (pos < top) {
            ls[i] = sta[pos + 1];
        }
        sta[++pos] = i;
        top = pos;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }
    build();
    long long ans1 = 0, ans2 = 0;
    for (int i = 1; i <= n; i++) {
        ans1 ^= 1LL * i * (ls[i] + 1);
        ans2 ^= 1LL * i * (rs[i] + 1);
    }
    cout << ans1 << " " << ans2 << endl;
    return 0;
}
```

图\割边与割边缩点（EBCC）

图\割边与割边缩点（EBCC）\割边与割边缩点（EBCC）.cpp

```
std::set<std::pair<int, int>> E;

struct EBCC {
    int n;
    std::vector<std::vector<int>> adj;
    std::vector<int> stk;
    std::vector<int> dfn, low, bel;
    int cur, cnt;

    EBCC() {}
    EBCC(int n) {
        init(n);
    }

    void init(int n) {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.resize(n);
        bel.assign(n, -1);
        stk.clear();
        cur = cnt = 0;
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void dfs(int x, int p) {
        dfn[x] = low[x] = cur++;
        stk.push_back(x);

        for (auto y : adj[x]) {
            if (y == p) {
                continue;
            }
            if (dfn[y] == -1) {
                E.emplace(x, y);
                dfs(y, x);
                low[x] = std::min(low[x], low[y]);
            } else if (bel[y] == -1 && dfn[y] < dfn[x]) {
                E.emplace(x, y);
                low[x] = std::min(low[x], dfn[y]);
            }
        }

        if (dfn[x] == low[x]) {
            int y;
            do {
                y = stk.back();
                bel[y] = cnt;
                stk.pop_back();
            } while (y != x);
            cnt++;
        }
    }

    std::vector<int> work() {
        dfs(0, -1);
        return bel;
    }

    struct Graph {
        int n;
        std::vector<std::pair<int, int>> edges;
        std::vector<int> siz;
        std::vector<int> cnte;
    };
    Graph compress() {
        Graph g;
        g.n = cnt;
        g.siz.resize(cnt);
        g.cnte.resize(cnt);
        for (int i = 0; i < n; i++) {
            g.siz[bel[i]]++;
            for (auto j : adj[i]) {
                if (bel[i] < bel[j]) {
                    g.edges.emplace_back(bel[i], bel[j]);
                } else if (i < j) {
```

```
                    g.cnte[bel[i]]++;
                }
            }
        }
        return g;
    }
};
```

图\强连通分量缩点（SCC）

图\强连通分量缩点（SCC）\强连通分量缩点（SCC）.cpp

```cpp
struct SCC {
    int n;                              // 图的顶点数
    std::vector<std::vector<int>> adj;  // 邻接表存储图
    std::vector<int> stk;               // 用于DFS的栈
    std::vector<int> dfn;               // 每个节点的DFS序（被访问的时间戳）
    std::vector<int> low;               // 通过回溯能到达的最小DFS序
    std::vector<int> bel;               // 每个节点所属的SCC编号（分量id）
    int cur, cnt;                       // cur：当前时间戳计数，cnt：SCC计数器

    SCC() {}
    SCC(int n) { init(n); }

    void init(int n) {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.resize(n);
        bel.assign(n, -1);
        stk.clear();
        cur = cnt = 0;
    }

    void addEdge(int u, int v) { adj[u].push_back(v); }

    void dfs(int x) {
        dfn[x] = low[x] = cur++;
        stk.push_back(x);

        for (auto y : adj[x]) {
            if (dfn[y] == -1) {
                dfs(y);
                low[x] = std::min(low[x], low[y]);
            } else if (bel[y] == -1) {
                low[x] = std::min(low[x], dfn[y]);
            }
        }

        if (dfn[x] == low[x]) {
            int y;
            do {
                y = stk.back();
                bel[y] = cnt;
                stk.pop_back();
            } while (y != x);
            cnt++;
        }
    }

    std::vector<int> work() {
        for (int i = 0; i < n; i++) {
            if (dfn[i] == -1) {
                dfs(i);
            }
        }
        return bel;
    }
};
```

图\最大匹配

图\最大匹配\一般图最大匹配.cpp

```cpp
struct Graph {
    int n;
    std::vector<std::vector<int>> e;
    Graph(int n) : n(n), e(n) {}
    void addEdge(int u, int v) {
        e[u].push_back(v);
        e[v].push_back(u);
    }
    std::vector<int> findMatching(int m, const auto &init) {
        std::vector<int> match(n, -1), vis(n), link(n), f(n), dep(n);
        for (auto [x, y] : init) {
            match[x] = y;
            match[y] = x;
        }
        // disjoint set union
        auto find = [&](int u) {
            while (f[u] != u) u = f[u] = f[f[u]];
            return u;
        };
        auto lca = [&](int u, int v) {
            u = find(u);
            v = find(v);
            while (u != v) {
                if (dep[u] < dep[v]) std::swap(u, v);
                u = find(link[match[u]]);
            }
            return u;
        };
        std::queue<int> que;
        auto blossom = [&](int u, int v, int p) {
            while (find(u) != p) {
                link[u] = v;
                v = match[u];
                if (vis[v] == 0) {
                    vis[v] = 1;
                    que.push(v);
                }
                f[u] = f[v] = p;
                u = link[v];
            }
        };
        // find an augmenting path starting from u and augment (if exist)
        auto augment = [&](int u) {
            while (!que.empty()) que.pop();
            std::iota(f.begin(), f.end(), 0);
            // vis = 0 corresponds to inner vertices, vis = 1 corresponds to outer vertices
            std::fill(vis.begin(), vis.end(), -1);
            que.push(u);
            vis[u] = 1;
            dep[u] = 0;
            int y = -1;
            while (!que.empty()) {
                int u = que.front();
                que.pop();
                if (u >= m) {
                    y = u;
                }
                for (auto v : e[u]) {
                    if (vis[v] == -1) {
                        vis[v] = 0;
                        link[v] = u;
                        dep[v] = dep[u] + 1;
                        // found an augmenting path
                        if (match[v] == -1) {
                            for (int x = v, y = u, temp; y != -1; x = temp, y = x == -1 ? -1 : link[x]) {
                                temp = match[y];
                                match[x] = y;
                                match[y] = x;
                            }
                            return;
                        }
                        vis[match[v]] = 1;
                        dep[match[v]] = dep[u] + 2;
                        que.push(match[v]);
                    } else if (vis[v] == 1 && find(v) != find(u)) {
                        // found a blossom
                        int p = lca(u, v);
                        blossom(u, v, p);
                        blossom(v, u, p);
                    }
```

```
                }
            }
            if (y != -1) {
                for (int x = -1, temp; y != -1; x = temp, y = x == -1 ? -1 : link[x]) {
                    temp = match[y];
                    if (x != -1) {
                        match[x] = y;
                    }
                    match[y] = x;
                }
            }
        };
        for (int u = 0; u < m; ++u)
            if (match[u] == -1) augment(u);
        return match;
    }
};
/*
  带花树算法（Edmonds’ Blossom Algorithm）—— 一般图最大匹配模板

  使用方法：
    Graph g(n);                        // 创建一个 n 个点的无向图
    g.addEdge(u, v);                   // 添加一条无向边 (u, v)
    auto match = g.findMatching(n, {}); // 求最大匹配，n 表示起始点范围（一般为 n），初始匹配为空

    // 结果处理
    for (int i = 0; i < n; ++i)
        if (match[i] != -1 && i < match[i])
            std::cout << i << " " << match[i] << "\n";

  --------------------------------
  主要接口：

  1. 初始化
    Graph(int n);
    @param n 节点数 (int)
    @effect 初始化一个 n 个点的无向图，编号为 [0, n)

  2. 添加边
    void addEdge(int u, int v);
    @param u, v 无向边的两个端点 (int)
    @effect 在图中添加边 (u, v)，自动双向存储

  3. 求最大匹配
    std::vector<int> findMatching(int m, const auto &init);
    @param m 起始点范围，一般取 n；若为二分图可设为左部点数
    @param init 初始匹配边集，如 {{u1, v1}, {u2, v2}}，可为空 {}
    @return 长度为 n 的数组 match，match[i] = j 表示 i 与 j 匹配，-1 表示未匹配
    @effect 运行带花树算法，返回一般图的最大匹配结果

  4. 输出匹配边集
    遍历结果时需注意匹配对称性，推荐输出方式：
      for (int i = 0; i < n; ++i)
          if (i < match[i])
              std::cout << i << " " << match[i] << "\n";

  5. 匹配大小
    int matching_size = count_if(match.begin(), match.end(), [&](int v){ return v > -1; }) / 2;

  --------------------------------
  算法说明：
  - 该算法实现 Edmonds 的带花树算法 (Blossom Algorithm)
  - 可处理任意无向图（含奇环），时间复杂度 O(n^3)
  - 自动维护并查集、层次图、花缩合等结构
  - 支持在已有匹配基础上继续扩展（通过 init 参数）
  - 对于二分图匹配，仅需将 m 设置为左侧点数即可

*/
```

图\最大流

图\最大流\Maxflow.cpp

```cpp
constexpr int inf = 1E9;
template <class T>
struct MaxFlow {
    struct _Edge {
        int to;
        T cap;
        _Edge(int to, T cap) : to(to), cap(cap) {}
    };

    int n;
    std::vector<_Edge> e;
    std::vector<std::vector<int>> g;
    std::vector<int> cur, h;

    MaxFlow() {}
    MaxFlow(int n) { init(n); }

    void init(int n) {
        this->n = n;
        e.clear();
        g.assign(n, {});
        cur.resize(n);
        h.resize(n);
    }

    bool bfs(int s, int t) {
        h.assign(n, -1);
        std::queue<int> que;
        h[s] = 0;
        que.push(s);
        while (!que.empty()) {
            const int u = que.front();
            que.pop();
            for (int i : g[u]) {
                auto [v, c] = e[i];
                if (c > 0 && h[v] == -1) {
                    h[v] = h[u] + 1;
                    if (v == t) {
                        return true;
                    }
                    que.push(v);
                }
            }
        }
        return false;
    }

    T dfs(int u, int t, T f) {
        if (u == t) {
            return f;
        }
        auto r = f;
        for (int &i = cur[u]; i < int(g[u].size()); ++i) {
            const int j = g[u][i];
            auto [v, c] = e[j];
            if (c > 0 && h[v] == h[u] + 1) {
                auto a = dfs(v, t, std::min(r, c));
                e[j].cap -= a;
                e[j ^ 1].cap += a;
                r -= a;
                if (r == 0) {
                    return f;
                }
            }
        }
        return f - r;
    }
    void addEdge(int u, int v, T c) {
        g[u].push_back(e.size());
        e.emplace_back(v, c);
        g[v].push_back(e.size());
        e.emplace_back(u, 0);
    }
    T flow(int s, int t) {
        T ans = 0;
        while (bfs(s, t)) {
            cur.assign(n, 0);
            ans += dfs(s, t, std::numeric_limits<T>::max());
        }
        return ans;
```

```
    }

    std::vector<bool> minCut() {
        std::vector<bool> c(n);
        for (int i = 0; i < n; i++) {
            c[i] = (h[i] != -1);
        }
        return c;
    }

    struct Edge {
        int from;
        int to;
        T cap;
        T flow;
    };
    std::vector<Edge> edges() {
        std::vector<Edge> a;
        for (int i = 0; i < e.size(); i += 2) {
            Edge x;
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.flow = e[i + 1].cap;
            a.push_back(x);
        }
        return a;
    }
};

/*
  Dinic 最大流模板
  使用方法:
    MaxFlow<int> mf(n);          // 创建一个支持 int 容量的最大流对象，点数 n
    mf.addEdge(u, v, cap);       // 添加边
    auto ans = mf.flow(s, t);    // 求解从 s 到 t 的最大流
    auto cut = mf.minCut();      // 得到最小割的可达点集
    auto es = mf.edges();        // 得到所有边的 (from, to, cap, flow)

  -------------------------------
  主要接口:

  1. 初始化
    void init(int n);
    @param n 节点数 (int)
    @effect 初始化一个 n 个节点的空图

  2. 添加边
    void addEdge(int u, int v, T c);
    @param u 起点 (int) / v 终点 (int) / c 容量 (T)
    @effect 添加容量为 c 的有向边 u->v，并自动加反向边 (v->u, cap=0)

  3. 求最大流
    T flow(int s, int t);
    @param s 源点 (int) / t 汇点 (int)
    @return 最大流的流量值 (T)
    @effect 运行 Dinic 算法，求解 s-t 最大流

  4. 最小割
    std::vector<bool> minCut();
    @return bool 数组 (size=n)，c[i]=true 表示节点 i 在 s 可达集合中
    @effect 调用 flow() 后使用，返回最小割对应的点集划分

  5. 边信息
    struct Edge {
      int from;  @field 边的起点 (int)
      int to;    @field 边的终点 (int)
      T cap;     @field 边的总容量 (T)
      T flow;    @field 已经流过的流量 (T)
    };

    std::vector<Edge> edges();
    @return 返回所有原始边（不含反向边）的列表 (vector<Edge>)
    @effect 遍历存图时成对的正/反边，合并为一条包含 from/to/cap/flow 的边
*/
```

字符串\SAM(后缀自动机)

字符串\SAM(后缀自动机)\SAM.cpp

```cpp
struct SAM {
    static constexpr int ALPHABET_SIZE = 26;
    struct Node {
        int len;
        int link;
        std::array<int, ALPHABET_SIZE> next;
        Node() : len{}, link{}, next{} {}
    };
    std::vector<Node> t;
    SAM() {
        init();
    }
    void init() {
        t.assign(2, Node());
        t[0].next.fill(1);
        t[0].len = -1;
    }
    int newNode() {
        t.emplace_back();
        return t.size() - 1;
    }
    int extend(int p, int c) {
        if (t[p].next[c]) {
            int q = t[p].next[c];
            if (t[q].len == t[p].len + 1) {
                return q;
            }
            int r = newNode();
            t[r].len = t[p].len + 1;
            t[r].link = t[q].link;
            t[r].next = t[q].next;
            t[q].link = r;
            while (t[p].next[c] == q) {
                t[p].next[c] = r;
                p = t[p].link;
            }
            return r;
        }
        int cur = newNode();
        t[cur].len = t[p].len + 1;
        while (!t[p].next[c]) {
            t[p].next[c] = cur;
            p = t[p].link;
        }
        t[cur].link = extend(p, c);
        return cur;
    }
    int extend(int p, char c, char offset = 'a') {
        return extend(p, c - offset);
    }

    int next(int p, int x) {
        return t[p].next[x];
    }

    int next(int p, char c, char offset = 'a') {
        return next(p, c - 'a');
    }

    int link(int p) {
        return t[p].link;
    }

    int len(int p) {
        return t[p].len;
    }

    int size() {
        return t.size();
    }
};
/*
// -------- 1. 构建与基础操作 --------
sam.init();                          // void: 初始化，设置节点0为哨兵，节点1为Root
int cur = sam.extend(last, c);       // int:  插入字符 c，必须传入上一次的节点 last。返回当前新节点编号
int cur = sam.extend(last, c, 'a'); // int:  同上，支持由字符自动转为偏移量 (c - 'a')

// -------- 2. 节点属性访问 (API封装) --------
int l = sam.len(u);                  // int:  获取节点 u 的最大子串长度 (maxlen)
int p = sam.link(u);                 // int:  获取节点 u 的后缀链接 (Parent/Fail指针)
```

```
int v = sam.next(u, c);             // int:  获取 DAG 转移边，无路返回 0 (注意不是-1)

// -------- 3. 内部直接访问 (手写逻辑常用) --------
sam.t[u].len;                       // 变量: maxlen
sam.t[u].link;                      // 变量: parent link
sam.t[u].next[c];                   // 数组: 转移边 (0表示空)

// -------- 4. 实战核心约定 (Crucial!) --------
// [节点索引]: 0 是哨兵(Dummy)，1 是根节点(Root/空串)，有效节点从 1 开始。
// [遍历所有]: for(int i = 1; i < sam.size(); i++) { ... }
// [最短长度]: minlen(u) = sam.t[sam.t[u].link].len + 1
// [本质不同]: sum(len(u) - len(link(u)))  (u > 1)

// -------- 5. 标准构建用法示例 --------
SAM sam;
int last = 1; // !重要: 初始状态必须是 1
for (char c : s) {
    last = sam.extend(last, c);
}
*/
```

字符串\SAM(后缀自动机)\my_SAM.cpp

```cpp
struct SAM {
    // --- 配置项 ---
    static constexpr int ALPHABET_SIZE = 26;
    static constexpr int OFFSET = 'a';

    // --- 节点结构体 ---
    struct Node {
        int len;                              // r(u): maxlen
        int link;                             // parent (suffix link)
        std::array<int, ALPHABET_SIZE> next;  // DAG 边 (大字符集请换成 map)
        int cnt_init;                         // 初始标记: 1=实点, 0=克隆

        Node() : len(0), link(0), cnt_init(0) { next.fill(0); }
    };

    // --- 公开成员 ---
    std::vector<Node> t;      // 节点池 (有效下标 1 ~ size()-1)
    int last;                 // 当前光标位置
    long long uniq_sub;       // 【核心】实时维护本质不同子串总数

    // --- 统计数据 ---
    std::vector<int> CNT;     // 内部出现次数数组 (需 build)
    std::vector<int> tax, rk; // 辅助内存 (基数排序用)

   public:
    // --- 构造与初始化 ---
    SAM() { init(); }
    SAM(int n) { init(n); }

    void init(int n = 0) {
        t.clear();
        CNT.clear();
        if (n > 0) {
            t.reserve(n * 2 + 1);
            CNT.reserve(n * 2 + 1);
            tax.reserve(n + 1);
            rk.reserve(n * 2 + 1);
        }
        t.assign(2, Node());  // 0:哨兵, 1:根
        t[0].next.fill(1);
        t[0].len = -1;
        last = 1;
        uniq_sub = 0;         // 重置计数器
    }

    // --- 核心操作 ---

    // 【广义 SAM 关键】: 插入新串前必须调用, 重置光标到根
    void restart() { last = 1; }

    int extend(int c_int) {
        int p = last;

        // [分支1] 路径已存在 (广义 SAM 特有或重复子串)
        // 此时不产生新的本质不同子串, 仅做路径复用或拆分
        if (t[p].next[c_int]) {
            int q = t[p].next[c_int];
            if (t[q].len == t[p].len + 1) {
                last = q; // 状态完全匹配, 直接复用
            } else {
                // 状态被包含, 需要拆分 (Clone)
                int r = newNode();
                t[r].len = t[p].len + 1;
                t[r].link = t[q].link;
                t[r].next = t[q].next;
                t[r].cnt_init = 0; // 克隆点无初始贡献
                t[q].link = r;
                while (t[p].next[c_int] == q) {
                    t[p].next[c_int] = r;
                    p = t[p].link;
                }
                last = r;
            }
            // 注意: 分支1 只是维护结构, 不更新 uniq_sub
            return last;
        }

        // [分支2] 创建新实点 (Cur)
        // 只有此处代表产生了新的子串集合
        int cur = newNode();
```

```
        t[cur].len = t[p].len + 1;
        t[cur].cnt_init = 1; // 实点有初始贡献

        while (!t[p].next[c_int]) {
            t[p].next[c_int] = cur;
            p = t[p].link;
        }

        // 寻找 link
        int link_node;
        if (t[p].next[c_int]) {
            int q = t[p].next[c_int];
            if (t[q].len == t[p].len + 1) {
                link_node = q;
            } else {
                int r = newNode();
                t[r].len = t[p].len + 1;
                t[r].link = t[q].link;
                t[r].next = t[q].next;
                t[r].cnt_init = 0;
                t[q].link = r;
                while (t[p].next[c_int] == q) {
                    t[p].next[c_int] = r;
                    p = t[p].link;
                }
                link_node = r;
            }
        } else {
            link_node = 1;
        }
        t[cur].link = link_node;

        // 【核心统计】更新本质不同子串数
        // 增量 = len(cur) - len(link(cur))
        uniq_sub += (long long)t[cur].len - t[t[cur].link].len;

        last = cur;
        return cur;
    }

    int extend(char c) { return extend(c - OFFSET); }
    void extend(const std::string& s) { for (char c : s) extend(c); }

    // --- 统计与构建 (O(N)) ---
    // 注意：如果只求 uniq_sub，不需要调用 build()
    // 只有需要查询 cnt(u) 或 query(string) 时才调用
    void build() {
        int n = t.size();
        CNT.assign(n, 0);

        // 1. 初始贡献
        for (int i = 0; i < n; i++) CNT[i] = t[i].cnt_init;

        // 2. 基数排序
        // 为了安全覆盖所有可能的长度，这里用 n 作为桶的大小
        if (tax.size() <= n) tax.assign(n + 1, 0);
        else std::fill(tax.begin(), tax.begin() + n + 1, 0);

        if (rk.size() < n) rk.resize(n);

        for (int i = 1; i < n; i++) tax[t[i].len]++;
        for (int i = 1; i <= n; i++) tax[i] += tax[i - 1];
        for (int i = 1; i < n; i++) rk[tax[t[i].len]--] = i;

        // 3. 拓扑累加
        for (int i = n - 1; i >= 1; i--) {
            int u = rk[i];
            if (t[u].link > 1) CNT[t[u].link] += CNT[u];
        }
    }

    // --- 统一查询接口 ---

    // 获取实时本质不同子串数 (无需 build)
    long long get_uniq_sub() const {
        return uniq_sub;
    }

    // 获取节点 u 的出现次数 (需 build)
    long long cnt(int u) const {
        return CNT[u];
    }

    // 查特定字符串的出现次数 (需 build)
```

```
    long long query(const std::string& T) const {
        int u = run(T);
        return u ? CNT[u] : 0;
    }

    // --- 基础 API ---
    int run(const std::string& T) const {
        int u = 1;
        for (char c : T) {
            int c_int = c - OFFSET;
            if (!t[u].next[c_int]) return 0;
            u = t[u].next[c_int];
        }
        return u;
    }

    int size() const { return t.size(); }
    int link(int u) const { return t[u].link; }
    int r(int u) const { return t[u].len; }
    int l(int u) const { return t[t[u].link].len + 1; }
    int next(int u, char c) const { return t[u].next[c - OFFSET]; }
    int next(int u, int c) const { return t[u].next[c]; }

   private:
    int newNode() {
        t.emplace_back();
        return t.size() - 1;
    }
};

/* ========================================================================
   SAM 模版接口说明与使用指南
========================================================================

========================================================================
一、构建与初始化
------------------------------------------------------------------------
void init(int n = 0)
   - 作用：初始化自动机，清空节点池和计数器。
   - 参数 n：(可选) 预估字符串长度，用于 reserve 内存。

void restart()
   - 作用：【广义 SAM 专用】将 last 指针重置为 1 (根节点)。
   - 场景：在插入多个字符串时，每处理完一个串调用一次，再插入下一个。

int extend(char c) / void extend(string s)
   - 作用：插入字符或字符串，增量构建 SAM。
   - 机制：内部会实时更新 uniq_sub (本质不同子串数)。

void build()
   - 作用：【必须显式调用】对 SAM 进行基数排序和拓扑更新，计算所有节点的 CNT。
   - 时机：在所有字符串 extend 完毕后调用。
   - 复杂度：O(N)。

========================================================================
二、DAG (自动机图) 维度查询
------------------------------------------------------------------------
int next(int u, char c) / int next(int u, int c)
   - 作用：查询转移边。支持传字符('a')或数字(0)。

int run(const std::string& s)
   - 作用：在自动机上跑字符串 s，返回最终停留的节点编号。
   - 返回：若中途失配返回 0。

long long query(const std::string& T)
   - 作用：查询 T 在原串中的出现次数。需先 build()。

========================================================================
三、Parent Tree (后缀链接树) 维度查询
------------------------------------------------------------------------
int link(int u)
   - 作用：获取父节点 (Suffix Link)。

int r(int u)
   - 作用：获取节点 u 代表的最长子串长度 (Max Length)。

int l(int u)
   - 作用：获取节点 u 代表的最短子串长度 (Min Length)。
   - 公式：sam.r(sam.link(u)) + 1。
   - 解释：节点 u 代表的子串长度区间为 [l(u), r(u)]。

========================================================================
四、数据获取接口 (Data Access)
------------------------------------------------------------------------
```

```
long long cnt(int u)
   - 作用：获取节点 u 代表子串的出现次数。
   - 前置：必须先 build()。

long long get_uniq_sub()
   - 作用：获取本质不同的子串总数。
   - 特性：在线算法，extend 过程中实时更新，无需 build。

==========================================================================
五、API 速查 (Signature - Usage)
--------------------------------------------------------------------------
SAM sam(n);      // void - 初始化SAM，n为预估大小
sam.init(n);     // void - 初始化SAM，n为预估大小
sam.restart();   // void - 重置插入点last=1，用于广义SAM换串
sam.extend(c);   // int - 插入字符c，返回节点编号
sam.extend(s);   // void - 插入字符串s
sam.build();     // void - 构建统计信息(CNT)，必须最后显式调用

sam.cnt(u);      // long long - 获取节点u的出现次数(需build)
sam.get_uniq_sub(); // long long - 获取本质不同子串总数(在线，无需build)
sam.query(T);    // long long - 查询字符串T的出现次数(需build)
sam.run(s);      // int - 在SAM上运行s，返回节点编号或0

sam.next(u, c);  // int - 获取u经过c转移到的节点编号，如果不存在返回0
sam.link(u);     // int - 获取u的后缀链接(父节点)
sam.r(u);        // int - 获取u的最长子串长度(maxlen)
sam.l(u);        // int - 获取u的最短子串长度(minlen)
sam.size();      // int - 获取SAM总节点数(有效下标1~size-1)
==========================================================================
*/
```

字符串\SAM(后缀自动机)\my_SAM_map.cpp

```cpp
/*
    SAM Template (Map Version)
    适用场景: 字符集很大 (如 10^9) 或 未知字符集
    特性:
    1. 广义 SAM 支持 (restart)
    2. 在线统计本质不同子串 (uniq_sub)
    3. std::map 处理转移边
*/
struct SAM {
    // --- 节点结构体 ---
    struct Node {
        int len;                  // r(u): maxlen
        int link;                 // parent (suffix link)
        std::map<int, int> next;  // 【关键】使用 map 处理大字符集
        int cnt_init;             // 初始标记: 1=实点, 0=克隆

        Node() : len(0), link(0), cnt_init(0) {}
    };

    // --- 公开成员 ---
    std::vector<Node> t;      // 节点池 (有效下标 1 ~ size()-1)
    int last;                 // 当前光标位置
    long long uniq_sub;       // 【核心】实时维护本质不同子串总数

   private:
    std::vector<int> CNT;     // 内部出现次数数组 (需 build)
    std::vector<int> tax, rk; // 辅助内存

   public:
    // --- 构造与初始化 ---
    SAM() { init(); }
    SAM(int n) { init(n); }

    void init(int n = 0) {
        t.clear();
        CNT.clear();
        if (n > 0) {
            t.reserve(n * 2 + 1);
            CNT.reserve(n * 2 + 1);
            tax.reserve(n + 1);
            rk.reserve(n * 2 + 1);
        }
        t.emplace_back(); // 0: 哨兵
        t.emplace_back(); // 1: 根
        t[0].len = -1;
        t[1].len = 0;
        t[1].link = 0;
        last = 1;
        uniq_sub = 0;
    }

    // --- 核心操作 ---

    // 【广义 SAM 关键】: 插入新串前调用, 重置光标到根
    void restart() { last = 1; }

    // 【核心构建逻辑】
    // 返回值: 当前的本质不同子串总数
    long long extend(int c) {
        int p = last;

        // [分支1] 路径已存在 (广义 SAM 特有或重复子串)
        // 此时不产生新的本质不同子串, 仅做路径复用或拆分
        if (t[p].next.count(c)) {
            int q = t[p].next[c];
            if (t[q].len == t[p].len + 1) {
                last = q; // 完全复用
            } else {
                // 状态被包含, 需要拆分 (Clone)
                int r = newNode();
                t[r].len = t[p].len + 1;
                t[r].link = t[q].link;
                t[r].next = t[q].next; // map 拷贝
                t[r].cnt_init = 0;     // 克隆点无初始贡献
                t[q].link = r;
                while (p && t[p].next.count(c) && t[p].next[c] == q) {
                    t[p].next[c] = r;
                    p = t[p].link;
                }
                last = r;
```

```
        }
        // 分支1 不更新 uniq_sub
        return uniq_sub;
    }

    // [分支2] 创建新实点 (Cur)
    // 只有此处代表产生了新的子串集合
    int cur = newNode();
    t[cur].len = t[p].len + 1;
    t[cur].cnt_init = 1; // 实点有初始贡献

    while (p && !t[p].next.count(c)) {
        t[p].next[c] = cur;
        p = t[p].link;
    }

    if (!p) {
        t[cur].link = 1;
    } else {
        int q = t[p].next[c];
        if (t[q].len == t[p].len + 1) {
            t[cur].link = q;
        } else {
            int r = newNode();
            t[r].len = t[p].len + 1;
            t[r].link = t[q].link;
            t[r].next = t[q].next; // map 拷贝
            t[r].cnt_init = 0;
            t[q].link = r;
            t[cur].link = r;
            while (p && t[p].next.count(c) && t[p].next[c] == q) {
                t[p].next[c] = r;
                p = t[p].link;
            }
        }
    }

    // 【核心统计】更新本质不同子串数
    // 增量 = len(cur) - len(link(cur))
    uniq_sub += (long long)t[cur].len - t[t[cur].link].len;

    last = cur;
    return uniq_sub;
}

// 辅助重载
void extend(const std::vector<int>& s) { for (int c : s) extend(c); }
void extend(const std::string& s) { for (char c : s) extend(c); }

// --- 统计与构建 (O(N)) ---
// 只有需要查询 cnt(u) 或 query(string) 时才调用
void build() {
    int n = t.size();
    CNT.assign(n, 0);

    // 1. 初始贡献
    for (int i = 0; i < n; i++) CNT[i] = t[i].cnt_init;

    // 2. 基数排序
    if (tax.size() <= n) tax.assign(n + 1, 0);
    else std::fill(tax.begin(), tax.begin() + n + 1, 0);

    if (rk.size() < n) rk.resize(n);

    for (int i = 1; i < n; i++) tax[t[i].len]++;
    for (int i = 1; i <= n; i++) tax[i] += tax[i - 1];
    for (int i = 1; i < n; i++) rk[tax[t[i].len]--] = i;

    // 3. 拓扑累加
    for (int i = n - 1; i >= 1; i--) {
        int u = rk[i];
        if (t[u].link > 1) CNT[t[u].link] += CNT[u];
    }
}

// --- 统一查询接口 ---

long long get_uniq_sub() const { return uniq_sub; }
long long cnt(int u) const { return CNT[u]; }

// 泛型 Run: 支持 vector<int> 或 string
template <typename Container>
int run(const Container& T) const {
    int u = 1;
```

```
        for (auto c : T) {
            auto it = t[u].next.find(c);
            if (it == t[u].next.end()) return 0;
            u = it->second;
        }
        return u;
    }

    // 查询出现次数 (需先 build)
    long long query(const std::vector<int>& T) const {
        int u = run(T);
        return u ? CNT[u] : 0;
    }
    long long query(const std::string& T) const {
        int u = run(T);
        return u ? CNT[u] : 0;
    }

    // --- 基础 API ---
    int size() const { return t.size(); }
    int link(int u) const { return t[u].link; }
    int r(int u) const { return t[u].len; }
    int l(int u) const { return t[t[u].link].len + 1; }

    // 安全获取 next，不存在返回 0
    int next(int u, int c) const {
        auto it = t[u].next.find(c);
        return (it != t[u].next.end()) ? it->second : 0;
    }

   private:
    int newNode() {
        t.emplace_back();
        return t.size() - 1;
    }
};

/* ========================================================================
   SAM (Map Version) 模版接口说明与使用指南
========================================================================

========================================================================
一、构建与初始化
------------------------------------------------------------------------
void init(int n = 0)
   - 作用：初始化自动机，清空节点池和计数器。
   - 参数 n：(可选) 预估总长度，用于 reserve 内存。

void restart()
   - 作用：【广义 SAM 专用】将 last 指针重置为 1 (根节点)。
   - 场景：在插入多个字符串时，每处理完一个串调用一次，再插入下一个。

long long extend(int c)
   - 作用：插入一个字符(或数字)。
   - 返回：当前的本质不同子串总数 (uniq_sub)。
   - 复杂度：O(log Σ) per char，map 操作带来 log 开销。

void extend(vector<int> s) / void extend(string s)
   - 作用：批量插入整个序列。

void build()
   - 作用：【显式调用】对 SAM 进行基数排序和拓扑更新，计算 CNT。
   - 时机：只有在需要查询子串出现次数时才调用。
   - 复杂度：O(N)。

========================================================================
二、数据获取与查询 (Query)
------------------------------------------------------------------------
long long get_uniq_sub()
   - 作用：获取本质不同的子串总数。
   - 特性：在线算法 (Online)，extend 过程中实时更新，无需 build。

long long cnt(int u)
   - 作用：获取节点 u 代表子串的出现次数。
   - 前置：必须先 build()。

long long query(vector<int> T) / query(string T)
   - 作用：查询序列 T 在原串中的出现次数。
   - 前置：必须先 build()。
   - 返回：出现次数，若 T 不是子串返回 0。

int run(vector<int> T) / run(string T)
   - 作用：在自动机上跑序列 T。
   - 返回：最终停留的节点编号，若失配返回 0。
```

```
==========================================================================
三、API 速查 (Signature - Usage)
--------------------------------------------------------------------------
SAM sam(n);              // 构造函数，n 为预估大小
sam.init(n);             // 初始化/重置
sam.restart();           // 广义 SAM 换串时重置光标

sam.extend(100);         // 插入数字 100，返回当前 uniq_sub
sam.extend(vec);         // 插入 vector<int>
sam.build();             // 构建 CNT 数组 (统计用)

sam.get_uniq_sub();      // 获取本质不同子串总数 (long long)
sam.cnt(u);              // 获取节点 u 的出现次数 (long long)
sam.query(T);            // 查询 T 的出现次数 (long long)

sam.size();              // 获取总节点数 (int)
sam.link(u);             // 获取后缀链接/父节点 (int)
sam.r(u);                // 获取最长子串长度 len (int)
sam.l(u);                // 获取最短子串长度 minlen (int)
sam.next(u, c);          // 获取转移边 u->c，不存在返回 0 (int)


sam.t[u].len;            // 节点 u 的长度
sam.t[u].link;           // 节点 u 的父节点
sam.t[u].next[c];        // (不推荐直接访问 map) 节点 u 的转移边
sam.uniq_sub;            // 实时本质不同子串数
==========================================================================
*/
```

字符串\hash

字符串\hash\Hash_String.cpp

```cpp
int qpow(int a, int b, int m) {
    a %= m;
    int res = 1;
    while (b > 0) {
        if (b & 1) res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res % m;
}
int inverse(int a, int MOD) { return qpow(a, MOD - 2, MOD); }

constexpr int inf = 1e18;
int P = 131, M = 1e9 + 7;
std::vector<int> pw(2e6, 1);
/*

*/
void slu() {
    std::string s;
    std::cin >> s;
    std::vector<int> hash(s.size() + 1, 0);
    for (int i = 1; i <= s.size(); i++) {
        hash[i] = (hash[i - 1] + pw[i - 1] * s[i - 1]) % M;
    }

    auto cul = [&](int l, int r) {
        int k = (hash[r] - hash[l - 1] + M) % M;
        k = (k * inverse(pw[l - 1], M)) % M;
        return k;
    };

    int q;
    std::cin >> q;
    while (q--) {
        int l1, r1, l2, r2;
        cin >> l1 >> r1 >> l2 >> r2;
        if (cul(l1, r1) == cul(l2, r2)) {
            cout << "Yes\n";
        } else {
            cout << "No\n";
        }
    }
}

signed main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    for (int i = 1; i < pw.size(); i++) {
        pw[i] = (pw[i - 1] * P) % M;
    }

    int t = 1;
    // cin >> t;

    while (t--) slu();
    return 0;
}
```

字符串\hash\Hash_int.cpp

```cpp
typedef unsigned long long ull;
struct HashPair {
    ull h1, h2;
    // 重载 == 运算符
    bool operator==(const HashPair& other) const { return h1 == other.h1 && h2 == other.h2; }
    // 重载 != 运算符
    bool operator!=(const HashPair& other) const { return !(*this == other); }
};

struct RollingHash {
    static const ull MOD2 = 1e9 + 7;
    static ull BASE1, BASE2;
    static bool initialized;

    std::vector<ull> h1, p1, h2, p2;

    static void init_random_base() {
        if (initialized) return;
        std::mt19937_64 rng(std::chrono::steady_clock::now().time_since_epoch().count());
        BASE1 = (rng() % 5000) + 13331;
        BASE2 = (rng() % 5000) + 13331;
        initialized = true;
    }

    template <typename T>
    RollingHash(const std::vector<T>& arr) {
        init_random_base();
        int n = arr.size();  // 原数组长度

        h1.resize(n + 1);
        p1.resize(n + 1);
        h2.resize(n + 1);
        p2.resize(n + 1);
        p1[0] = 1;
        h1[0] = 0;
        p2[0] = 1;
        h2[0] = 0;

        for (int i = 0; i < n; i++) {
            // 使用 i % len 模拟循环
            ull val = (ull)arr[i]+ 1;

            p1[i + 1] = p1[i] * BASE1;
            h1[i + 1] = h1[i] * BASE1 + val;

            p2[i + 1] = (p2[i] * BASE2) % MOD2;
            h2[i + 1] = (h2[i] * BASE2 + val) % MOD2;
        }
    }

    // 修改为左闭右开区间 [l, r)
    HashPair query(int l, int r) {
        // h[r] 恰好代表前 r 个元素 (下标 0 到 r-1) 的哈希
        // h[l] 恰好代表前 l 个元素 (下标 0 到 l-1) 的哈希
        // 这里的下标直接对应哈希数组的索引，非常直观
        ull res1 = h1[r] - h1[l] * p1[r - l];
        ull res2 = (h2[r] - h2[l] * p2[r - l] % MOD2 + MOD2) % MOD2;

        return {res1, res2};
    }
};

// 静态成员初始化
ull RollingHash::BASE1 = 131;
ull RollingHash::BASE2 = 13331;
bool RollingHash::initialized = false;

/*
DOCUMENTATION:
RollingHash hasher(a);
HashPair query(l,r)
区间定义：左闭右开 [l, r)，0-based。内部进行了双倍扩容，l 和 r 可以超过原数组长度 len。
子段对应的 HashPair 对象，支持 == 和 != 比较。
*/
```

字符串\hash\二维哈希.cpp

```cpp
#include "bits/stdc++.h"
/*
关注b站<Feijoa_Li>谢谢喵
UID:248614274
https://space.bilibili.com/248614274
 */
using namespace std;

using u64 = unsigned long long;
using u32 = unsigned;
using u128 = unsigned __int128;
using i128 = __int128;

#define int long long
#define itn long long
#define Ist first
#define Ind second
#define O (int)0
#define endl "\n"
#define PII pair<int, int>
constexpr int inf = 1e18;

int qpow(int a, int b, int m) {
    a %= m;
    int res = 1;
    while (b > 0) {
        if (b & 1) res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res % m;
}

int MOD = 1e9 + 7;
int P = 131, Q = 13331;
int iP = qpow(P, MOD - 2, MOD);
int iQ = qpow(Q, MOD - 2, MOD);
int N = 1e3 + 7;

vector<int> pw(N, 1), qw(N, 1), ipw(N, 1), iqw(N, 1);
void init() {
    for (int i = 1; i < N; i++) {
        pw[i] = pw[i - 1] * P % MOD;
        qw[i] = qw[i - 1] * Q % MOD;
        ipw[i] = ipw[i - 1] * iP % MOD;
        iqw[i] = iqw[i - 1] * iQ % MOD;
    }
}
vector<vector<int>> buildHash(const vector<string> &s,const vector<int> &pw,const vector<int> &qw) {
    int n = s.size(), m = s[0].size();
    vector<vector<int>> H(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            H[i][j] = (H[i - 1][j] + H[i][j - 1] - H[i - 1][j - 1] + MOD) % MOD;
            int val = s[i - 1][j - 1];
            H[i][j] = (H[i][j] + 1LL * val * pw[i] % MOD * qw[j] % MOD) % MOD;
        }
    }
    return H;
}

int getHash(const vector<vector<int>> &H,const vector<int> &ipw,const vector<int> &iqw ,int x1, int y1, int x2, int
y2) {
    int h = (H[x2][y2] - H[x1 - 1][y2] - H[x2][y1 - 1] + H[x1 - 1][y1 - 1]) % MOD;
    h = (h + MOD) % MOD;
    h = h * ipw[x1] % MOD * iqw[y1] % MOD; // 归一化
    return h;
}

void slu() {
    int n, m, a, b;
    cin >> n >> m >> a >> b;
    vector<string> s(n);
    for (int i = 0; i < n; i++) {
        cin >> s[i];
    }

    auto H = buildHash(s, pw, qw);
    map<int, int> mp;
```

```
    for (int i = 1; i + a - 1 <= n; i++) {
        for (int j = 1; j + b - 1 <= m; j++) {
            int ir = i + a - 1, jr = j + b - 1;
            int qh = getHash(H, ipw, iqw,i, j, ir, jr);
            mp[qh]++;
        }
    }

    int q;
    cin >> q;
    while (q--) {
        vector<string> t(a);
        for (int i = 0; i < a; i++) {
            cin >> t[i];
        }
        auto T = buildHash(t,pw,qw);
        int th = getHash(T, ipw, iqw,1, 1, a, b);
        if (mp.contains(th)) {
            cout << 1 << endl;
        } else {
            cout << 0 << endl;
        }
    }
}

signed main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    int t = 1;
    // cin >> t;

    init();
    while (t--) slu();
    return 0;
}
```

数论\(类欧几里得)

数论\(类欧几里得)\Feijoa_Li.cpp

```cpp
/**
* 类欧几里得算法
 * 计算 \sum_{i=0}^{n-1} floor((a*i + b) / c)
 */
struct Euclidean {
    using i64 = long long;
    using i128 = __int128;

    // 内部递归逻辑：solve(a, b, c, n)
    static i128 solve(i64 a, i64 b, i64 c, i64 n) {
        if (n <= 0) return 0;
        i128 ans = 0;

        // 1. 提取 a / c 和 b / c 的整数部分
        if (a >= c) {
            ans += (i128)n * (n - 1) / 2 * (a / c);
            a %= c;
        }
        if (b >= c) {
            ans += (i128)n * (b / c);
            b %= c;
        }

        // 2. 边界情况
        i64 y_max = (a * n + b) / c;
        if (y_max == 0) return ans;

        // 3. 递归核心：坐标轴翻转变换
        i64 x_max = (y_max * c - b);
        ans += (i128)(n - (x_max + a - 1) / a) * y_max;
        ans += solve(c, (a - x_max % a) % a, a, y_max);

        return ans;
    }

    /**
     * @brief 对外接口 cul(a, b, c, n)
     * @param a 系数
     * @param b 偏移量
     * @param c 分母
     * @param n 迭代范围 [0, n)
     */
    static i64 cul(i64 a, i64 b, i64 c, i64 n) {
        if (n <= 0) return 0;
        i128 offset = 0;

        // 处理负数平移逻辑，保证 a_pos, b_pos >= 0
        i64 a_pos = (a % c + c) % c;
        i64 b_pos = (b % c + c) % c;

        // 补偿由于平移产生的差值（使用 i128 防止 n*(n-1) 溢出）
        offset -= (i128)n * (n - 1) / 2 * ((a_pos - a) / c);
        offset -= (i128)n * ((b_pos - b) / c);

        return (i64)(offset + solve(a_pos, b_pos, c, n));
    }
};

/*

计算 \sum_{i=0}^{n-1} floor((a*i + b) / c) -> { [0,n) :> (ai + b) / m 下取整 }
Euclidean::cul(a, b, c, n)
a 系数 / b 偏移量 / c 分母 / n 迭代范围 [0, n)

*/
```

数论\CRT(中国剩余定理)

数论\CRT(中国剩余定理)\CRT(中国剩余定理).cpp

```cpp
#include "bits/stdc++.h"
/*
________  _______       ___         ___      ________      ________
|\  _____\|\  ___ \     |\  \       |\  \    |\   __  \    |\   __  \
\ \  \__/ \ \   __/|    \ \  \      \ \  \   \ \  \|\  \   \ \  \|\  \
 \ \   __\ \ \  \_|/__   \ \  \   __ \ \  \   \ \  \\\  \   \ \   __  \
  \ \  \_|  \ \  \_|\ \   \ \  \ |\  \\\_\  \   \ \  \\\  \   \ \  \ \  \
   \ \__\    \ \_______\   \ \__\\ \_________\   \ \_______\   \ \__\ \__\
    \|__|     \|_______|    \|__| \|_________|    \|_______|    \|__|\|__|
 */
using namespace std;
#define int long long
#define endl "\n"
#define PII pair<int,int>
/*
https://www.luogu.com.cn/problem/P1495
https://www.luogu.com.cn/problem/P4777
https://www.luogu.com.cn/problem/P3868
https://www.luogu.com.cn/problem/P4774
*/
//CRT中国剩余定理
//x ≡ r (mod m)     m互质
struct CRT {
    using i64 = long long;

    std::vector<i64> r, m;

    // 支持初始化大小
    CRT(int n = 0) {
        r.resize(n);
        m.resize(n);
    }

    // 清空数据（用于多组数据）
    void clear() {
        r.clear();
        m.clear();
    }

    // 添加一个同余方程 x ≡ rem (mod mod)
    void add_equation(i64 mod, i64 rem) {
        m.push_back(mod);
        r.push_back(rem);
    }

    // 求 ax + by = gcd(a, b)
    static i64 exgcd(i64 a, i64 b, i64 &x, i64 &y) {
        if (b == 0) {
            x = 1, y = 0;
            return a;
        }
        i64 g = exgcd(b, a % b, y, x);
        y -= a / b * x;
        return g;
    }

    // 求 a 在 mod m 下的逆元，如果不存在返回 -1
    static i64 modInverse(i64 a, i64 mod) {
        i64 x, y;
        i64 g = exgcd(a, mod, x, y);
        if (g != 1) return -1;
        x %= mod;
        if (x < 0) x += mod;
        return x;
    }

    // 龟速乘法,防止溢出
    static i64 fast_mul(i64 a, i64 b, i64 mod) {
        a = (a % mod + mod) % mod;
        b = (b % mod + mod) % mod;
        i64 res = 0;
        while (b) {
            if (b & 1) res = (res + a) % mod;
            a = (a + a) % mod;
            b >>= 1;
        }
        return res;
    }

    // 求解：返回最小非负解 x，若无解返回 -1
```

```
    i64 solve() {
        int n = r.size();
        i64 M = 1;
        for (int i = 0; i < n; ++i) M *= m[i];

        i64 res = 0;
        for (int i = 0; i < n; ++i) {
            i64 Mi = M / m[i];
            i64 inv = modInverse(Mi, m[i]);
            if (inv == -1) return -1;
            i64 term = fast_mul(fast_mul(r[i], Mi, M), inv, M);
            res = (res + term) % M;
        }
        return (res + M) % M;
    }
};

//拓展: ki * x ≡ ri (mod mi)   且mi不互质
struct ExtendedCRT {
    using i64 = long long;
    using i128 = __int128;
    vector<i64> k, r, m;

    ExtendedCRT(int n = 0) {
        k.resize(n);
        r.resize(n);
        m.resize(n);
    }
    void clear() {
        k.clear(); r.clear(); m.clear();
    }

    // 添加方程 k*x ≡ rem (mod mod)
    void add_equation(i64 ki, i64 mod, i64 rem) {
        k.push_back(ki);
        m.push_back(mod);
        r.push_back(rem);
    }

    // 扩展欧几里得: ax+by=g
    static i64 exgcd(i64 a, i64 b, i64 &x, i64 &y) {
        if (b == 0) { x = 1; y = 0; return a; }
        i64 g = exgcd(b, a % b, y, x);
        y -= a / b * x;
        return g;
    }

    // 合并两个标准形式方程 x≡r1(mod m1) 和 x≡r2(mod m2)
    static bool merge(i64 r1, i64 m1, i64 r2, i64 m2, i64 &r0, i64 &m0) {
        r1 = (r1 % m1 + m1) % m1;
        r2 = (r2 % m2 + m2) % m2;
        i64 x, y;
        i64 g = exgcd(m1, m2, x, y);
        if ((r2 - r1) % g != 0) return false;
        i128 lcm = (i128)m1 / g * m2;
        i128 diff = (i128)(r2 - r1) / g;
        i128 mul = (diff * (x % (m2/g) + (m2/g)) ) % (m2/g);
        i128 res = (i128)r1 + mul * m1;
        r0 = (i64)((res % lcm + lcm) % lcm);
        m0 = (i64)lcm;
        return true;
    }

    // 求解返回最小非负 x，若无解返回 -1
    i64 solve() {
        int n = k.size();
        if (n == 0) return 0;
        // 先将 k*x≡r (mod m) 化为标准 x≡r' (mod m')
        vector<i64> rr(n), mm(n);
        for (int i = 0; i < n; ++i) {
            i64 ki = k[i], mi = m[i], ri = r[i];
            i64 x0, y0;
            i64 g = exgcd(ki, mi, x0, y0);
            if (ri % g != 0) return -1;
            // 化简
            ki /= g; mi /= g; ri /= g;
            // 求逆
            i64 inv = (x0 % mi + mi) % mi;
            // 标准余数
            rr[i] = (i128)inv * ri % mi;
            mm[i] = mi;
        }
        // 迭代合并
        i64 r0 = rr[0], m0 = mm[0];
```

```
        for (int i = 1; i < n; ++i) {
            i64 r1, m1;
            if (!merge(r0, m0, rr[i], mm[i], r1, m1)) return -1;
            r0 = r1; m0 = m1;
        }
        return r0;
    }
};

void slu() {
    int n;
    std::cin >> n;
    CRT crt(n); // 预设大小
    for (int i = 0; i < n; ++i) {
        std::cin >> crt.m[i] >> crt.r[i];
    }

    long long ans = crt.solve();
    if (ans == -1) std::cout << "No solution\n";
    else std::cout << ans << "\n";

    crt.clear(); // 如果有多组数据，可以复用
}

signed main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    int T = 1;
    // cin >> T;
    while (T--)slu();
    return 0;
}
```

数论\MOD

数论\MOD\MOD_jly.cpp

```cpp
template <class T>
constexpr T power(T a, i64 b) {
    T res = 1;
    for (; b; b /= 2, a *= a) {
        if (b % 2) {
            res *= a;
        }
    }
    return res;
}

constexpr i64 mul(i64 a, i64 b, i64 p) {
    i64 res = a * b - (i64)(1.L * a * b / p) * p;
    res %= p;
    if (res < 0) {
        res += p;
    }
    return res;
}
template <i64 P>
struct MLong {
    i64 x;
    constexpr MLong() : x{} {}
    constexpr MLong(i64 x) : x{norm(x % getMod())} {}

    static i64 Mod;
    constexpr static i64 getMod() {
        if (P > 0) {
            return P;
        } else {
            return Mod;
        }
    }
    constexpr static void setMod(i64 Mod_) { Mod = Mod_; }
    constexpr i64 norm(i64 x) const {
        if (x < 0) {
            x += getMod();
        }
        if (x >= getMod()) {
            x -= getMod();
        }
        return x;
    }
    constexpr i64 val() const { return x; }
    explicit constexpr operator i64() const { return x; }
    constexpr MLong operator-() const {
        MLong res;
        res.x = norm(getMod() - x);
        return res;
    }
    constexpr MLong inv() const {
        assert(x != 0);
        return power(*this, getMod() - 2);
    }
    constexpr MLong &operator*=(MLong rhs) & {
        x = mul(x, rhs.x, getMod());
        return *this;
    }
    constexpr MLong &operator+=(MLong rhs) & {
        x = norm(x + rhs.x);
        return *this;
    }
    constexpr MLong &operator-=(MLong rhs) & {
        x = norm(x - rhs.x);
        return *this;
    }
    constexpr MLong &operator/=(MLong rhs) & { return *this *= rhs.inv(); }
    friend constexpr MLong operator*(MLong lhs, MLong rhs) {
        MLong res = lhs;
        res *= rhs;
        return res;
    }
    friend constexpr MLong operator+(MLong lhs, MLong rhs) {
        MLong res = lhs;
        res += rhs;
        return res;
    }
    friend constexpr MLong operator-(MLong lhs, MLong rhs) {
        MLong res = lhs;
        res -= rhs;
```

```
        return res;
    }
    friend constexpr MLong operator/(MLong lhs, MLong rhs) {
        MLong res = lhs;
        res /= rhs;
        return res;
    }
    friend constexpr std::istream &operator>>(std::istream &is, MLong &a) {
        i64 v;
        is >> v;
        a = MLong(v);
        return is;
    }
    friend constexpr std::ostream &operator<<(std::ostream &os, const MLong &a) { return os << a.val(); }
    friend constexpr bool operator==(MLong lhs, MLong rhs) { return lhs.val() == rhs.val(); }
    friend constexpr bool operator!=(MLong lhs, MLong rhs) { return lhs.val() != rhs.val(); }
};

template <>
i64 MLong<0LL>::Mod = (i64)(1E18) + 9;

template <int P>
struct MInt {
    int x;
    constexpr MInt() : x{} {}
    constexpr MInt(i64 x) : x{norm(x % getMod())} {}

    static int Mod;
    constexpr static int getMod() {
        if (P > 0) {
            return P;
        } else {
            return Mod;
        }
    }
    constexpr static void setMod(int Mod_) { Mod = Mod_; }
    constexpr int norm(int x) const {
        if (x < 0) {
            x += getMod();
        }
        if (x >= getMod()) {
            x -= getMod();
        }
        return x;
    }
    constexpr int val() const { return x; }
    explicit constexpr operator int() const { return x; }
    constexpr MInt operator-() const {
        MInt res;
        res.x = norm(getMod() - x);
        return res;
    }
    constexpr MInt inv() const {
        assert(x != 0);
        return power(*this, getMod() - 2);
    }
    constexpr MInt &operator*=(MInt rhs) & {
        x = 1LL * x * rhs.x % getMod();
        return *this;
    }
    constexpr MInt &operator+=(MInt rhs) & {
        x = norm(x + rhs.x);
        return *this;
    }
    constexpr MInt &operator-=(MInt rhs) & {
        x = norm(x - rhs.x);
        return *this;
    }
    constexpr MInt &operator/=(MInt rhs) & { return *this *= rhs.inv(); }
    friend constexpr MInt operator*(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res *= rhs;
        return res;
    }
    friend constexpr MInt operator+(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res += rhs;
        return res;
    }
    friend constexpr MInt operator-(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res -= rhs;
        return res;
    }
```

```
    friend constexpr MInt operator/(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res /= rhs;
        return res;
    }
    friend constexpr std::istream &operator>>(std::istream &is, MInt &a) {
        i64 v;
        is >> v;
        a = MInt(v);
        return is;
    }
    friend constexpr std::ostream &operator<<(std::ostream &os, const MInt &a) { return os << a.val(); }
    friend constexpr bool operator==(MInt lhs, MInt rhs) { return lhs.val() == rhs.val(); }
    friend constexpr bool operator!=(MInt lhs, MInt rhs) { return lhs.val() != rhs.val(); }
};

template <>
int MInt<0>::Mod = MOD;

template <int V, int P>
constexpr MInt<P> CInv = MInt<P>(V).inv();

constexpr int P = MOD;
using Z = MInt<P>;

struct Comb {
    int n;
    std::vector<Z> _fac;
    std::vector<Z> _invfac;
    std::vector<Z> _inv;

    Comb() : n{0}, _fac{1}, _invfac{1}, _inv{0} {}
    Comb(int n) : Comb() { init(n); }

    void init(int m) {
        if (m <= n) return;
        _fac.resize(m + 1);
        _invfac.resize(m + 1);
        _inv.resize(m + 1);

        for (int i = n + 1; i <= m; i++) {
            _fac[i] = _fac[i - 1] * i;
        }
        _invfac[m] = _fac[m].inv();
        for (int i = m; i > n; i--) {
            _invfac[i - 1] = _invfac[i] * i;
            _inv[i] = _invfac[i] * _fac[i - 1];
        }
        n = m;
    }

    Z fac(int m) {
        if (m > n) init(2 * m);
        return _fac[m];
    }
    Z invfac(int m) {
        if (m > n) init(2 * m);
        return _invfac[m];
    }
    Z inv(int m) {
        if (m > n) init(2 * m);
        return _inv[m];
    }
    Z binom(int n, int m) {
        if (n < m || m < 0) return 0;
        return fac(n) * invfac(m) * invfac(n - m);
    }
    Z lucas(int n, int m) {
        if (m == 0) return 1;
        int mod = Z::getMod();
        int cur_n = n % mod;
        int cur_m = m % mod;

        if (this->n < cur_n) init(cur_n);
        return lucas(n / mod, m / mod) * binom(cur_n, cur_m);
    }
} comb;
/*
power(a, b);            // a^b（快速幂）
mul(a, b, p);           // 安全大数乘法（返回 a*b mod p，防溢出）

// MInt<P>（静态模数）
a.val();                // 取值
a.inv();                // 逆元
```

```
-a;                     // 取负
a + b; a - b;           // 加减
a * b; a / b;           // 乘除
a += b; a -= b;         // 复合加减
a *= b; a /= b;         // 复合乘除
a == b; a != b;         // 比较

// MLong<P>（大模数/可动态设置）
MLong<0>::setMod(M);    // 设置模数

// Comb（组合数）
comb.init(n);           // 初始化
comb.fac(n);            // n!
comb.inv(n);            // n 的逆元
comb.invfac(n);         // n! 的逆元
comb.binom(n, k);       // C(n, k) - 大模数 n < P - O(1)
comb.lucas(n, k);       // L(n, k) - 小模数 n > p - O(log_P(n))。
*/
```

数论\get_factors

数论\get_factors\get_factors.cpp

```
std::vector<std::pair<i64, int>> get_factors(i64 n) {
    std::vector<std::pair<i64, int>> res;
    if (n % 2 == 0) {
        int cnt = 0;
        while (n % 2 == 0) {
            n /= 2;
            cnt++;
        }
        res.emplace_back(2, cnt);
    }
    for (i64 i = 3; i * i <= n; i += 2) {
        if (n % i == 0) {
            int cnt = 0;
            while (n % i == 0) {
                n /= i;
                cnt++;
            }
            res.emplace_back(i, cnt);
        }
    }
    if (n > 1) res.emplace_back(n, 1);

    return res;
}// 返回值：vector<{质因子，该因子的次数}>
```

数论\isPrime

数论\isPrime\ Euler.cpp

```cpp
//筛选1-n
const int N = 5e6 + 5;//定义筛法计算的上限（1,000,005）
vector<int> pri;//存储素数的向量
bool not_prime[N];//标记非素数的布尔数组
int phi[N];//存储欧拉函数值的数组--->欧拉函数ϕ(n)表示小于等于n的正整数中与n互质的数的个数。
void pre(int n) {
    phi[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!not_prime[i]) {
            pri.push_back(i);
            phi[i] = i - 1;
        }
        for (int pri_j : pri) {
            if (i * pri_j > n) break;
            not_prime[i * pri_j] = true;
            if (i % pri_j == 0) {
                phi[i * pri_j] = phi[i] * pri_j;
                break;
            }
            phi[i * pri_j] = phi[i] * phi[pri_j];
        }
    }
}

std::vector<int> minp, primes;
//primes: 保存所有 ≤ n 的质数。
// minp [x]: 记录数 x 的最小质因子(minimum prime factor)。
void sieve(int n) {
    minp.assign(n + 1, 0);
    primes.clear();

    for (int i = 2; i <= n; i++) {
        if (minp[i] == 0) {
            minp[i] = i;
            primes.push_back(i);
        }

        for (auto p : primes) {
            if (i * p > n) {
                break;
            }
            minp[i * p] = p;
            if (p == minp[i]) {
                break;
            }
        }
    }
}
```

数论\isPrime\Eazy Version.cpp

```cpp
#include "bits/stdc++.h"
/*
_______  ______       ___         ___      _______      _______
|\  _____\|\  ___ \     |\  \       |\  \    |\   __  \    |\   __  \
\ \  \__/ \ \   __/|    \ \  \      \ \  \   \ \  \|\  \   \ \  \|\  \
 \ \   __\ \ \  \_|/__   \ \  \   __ \ \  \   \ \  \\\  \   \ \   __  \
  \ \  \_|  \ \  \_|\ \   \ \  \ |\  \\\_\  \   \ \  \\\  \   \ \  \ \  \
   \ \__\    \ \_______\   \ \__\\ \_______\   \ \_______\   \ \__\ \__\
    \|__|     \|_______|    \|__| \|_______|    \|_______|    \|__|\|__|
 */
using namespace std;
#define int long long
#define endl "\n"
#define PII pair<int,int>
bool isPrime(int n) {
    if (n <= 1) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

/*

*/
void slu() {
}

signed main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    int T = 1;
    // cin >> T;
    while (T--)slu();
    return 0;
}
```

数论\isPrime\Miller Rabin.cpp

```cpp
#include "bits/stdc++.h"
//判断n是否为素数，使用Miller-Rabin素性测试
/*
________  _______       ___        ___     ________     ________
|\  _____\|\  ___ \     |\  \       |\  \    |\   __  \    |\   __  \
\ \  \__/ \ \   __/|    \ \  \      \ \  \   \ \  \|\  \   \ \  \|\  \
 \ \   __\ \ \  \_|/__   \ \  \   __ \ \  \   \ \  \\\  \   \ \   __  \
  \ \  \_|  \ \  \_|\ \   \ \  \ |\  \\_\  \   \ \  \\\  \   \ \  \ \  \
   \ \__\    \ \_______\   \ \__\\ \_________\   \ \_______\   \ \__\ \__\
    \|__|     \|_______|    \|__| \|_________|    \|_______|    \|__|\|__|
 */
using namespace std;
#define int long long
#define endl "\n"
#define PII pair<int,int>
int qmul(int a,int b,int mod)//快速乘
{
    int c=(long double)a/mod*b;
    int res=(unsigned long long)a*b-(unsigned long long)c*mod;
    return (res+mod)%mod;
}
int qpow(int a,int n,int mod)//快速幂
{
    int res=1;
    while(n)
    {
        if(n&1) res=qmul(res,a,mod);
        a=qmul(a,a,mod);
        n>>=1;
    }
    return res;
}
bool MRtest(int n)//Miinter Rabin Test
{
    if(n<3||n%2==0) return n==2;//特判
    int u=n-1,t=0;
    while(u%2==0) u/=2,++t;
    int ud[]={2,325,9375,28178,450775,9780504,1795265022};
    for(int a:ud)
    {
        int v=qpow(a,u,n);
        if(v==1||v==n-1||v==0) continue;
        for(int j=1;j<=t;j++)
        {
            v=qmul(v,v,n);
            if(v==n-1&&j!=t){v=1;break;}//出现一个n-1，后面都是1，直接跳出
            if(v==1) return 0;//这里代表前面没有出现n-1这个解，二次检验失败
        }
        if(v!=1) return 0;//Fermat检验
    }
    return 1;
}
/*

*/
void slu() {
    int n;
    cin>>n;
    if(MRtest(n)) cout<<"Yes"<<endl;
    else cout<<"No"<<endl;
}

signed main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    int T = 1;
    cin >> T;
    while (T--)slu();
    return 0;
}
```

数论\qpow

数论\qpow\qpow.cpp

```
int qpow(int a, int b, int m) {
    int res = 1 % m;
    for (a %= m; b; b >>= 1, a = a * a % m)
        if (b & 1) res = res * a % m;
    return res;
}

int qmul(int a,int b,int mod)//快速乘
{
    int c=(long double)a/mod*b;
    int res=(unsigned long long)a*b-(unsigned long long)c*mod;
    return (res+mod)%mod;
}
int unqpow(int a,int n,int mod)//快速幂
{
    int res=1;
    while(n)
    {
        if(n&1) res=qmul(res,a,mod);
        a=qmul(a,a,mod);
        n>>=1;
    }
    return res;
}
```

数论\qpow\sqr.cpp

```
/*
      State Vector Transfer: M * [a_{i-1}, a_{i-2}, ..., a_{i-n}, i^2, i, 1]^T
a_{k} = res.mat[n - 1][0] -> M^k * [a_{n-1}, a_{n-2}, ..., a_{0}  , n^2, n, 1]^T
   Target  | a_{i-1} | a_{i-2} | ... | a_{i-n} |  i^2  |  i  |  1  |  Notes
   --------|---------|---------|-----|---------|-------|-----|-----|---------------------------
   a_i     |   c1    |   c2    | ... |   cn    |   r   |  q  |  p  | a_i = Σ(ci*a_{i-j}) + r*i^2 + q*i + p
   a_{i-1} |   1     |   0     | ... |   0     |   0   |  0  |  0  | shift
   a_{i-2} |   0     |   1     | ... |   0     |   0   |  0  |  0  | shift
   ...     |   ...   |   ...   | ... |   ...   |  ...  | ... | ... | ...
   (i+1)^2 |   0     |   0     | ... |   0     |   1   |  2  |  1  | (i+1)^2 = 1 * i^2 + 2 * i + 1
   (i+1)   |   0     |   0     | ... |   0     |   0   |  1  |  1  | (i+1)   = 0 * i^2 + 1 * i + 1
   1       |   0     |   0     | ... |   0     |   0   |  0  |  1  | 1       = 0 * i^2 + 0 * i + 1
*/
template <typename T>
struct Matrix {
    // 定义最半环(+ , *)
    // 标准矩阵乘法: C[i][j] = Σ (A[i][k] * B[k][j])
    struct Semiring {
        // Additive Identity: 加法的单位元 (e.g., 0 for +, -inf for max, 1 for *)
        static T add_id() { return 0; }
        // Multiplicative Identity: 乘法的单位元
        static T mul_id() { return 1; }

        static T add(T a, T b) { return (a + b) % MOD; }
        static T mul(T a, T b) { return (a * b) % MOD; }
    };

    int rows, cols;
    std::vector<T> mat;  // 一维扁平化存储

    Matrix(int r, int c) : rows(r), cols(c), mat(r * c, Semiring::add_id()) {}

    // 下标访问接口
    inline T& operator()(int r, int c) { return mat[r * cols + c]; }
    inline const T& operator()(int r, int c) const { return mat[r * cols + c]; }

    Matrix(std::initializer_list<std::initializer_list<T>> list) {
        rows = list.size();
        cols = (rows > 0) ? list.begin()->size() : 0;
        mat.assign(rows * cols, Semiring::add_id());
        int i = 0;
        for (const auto& row : list) {
            std::copy(row.begin(), row.begin() + std::min((int)row.size(), cols), mat.begin() + i * cols);
            i++;
        }
    }

    static Matrix identity(int n) {
        Matrix res(n, n);
        for (int i = 0; i < n; i++) res.mat[i * n + i] = Semiring::mul_id();
        return res;
    }

    Matrix operator*(const Matrix& other) const {
        assert(cols == other.rows);
        Matrix res(rows, other.cols);

        // 通用高性能 I-K-J 循环
        for (int i = 0; i < rows; i++) {
            int i_off = i * cols;
            int i_res_off = i * other.cols;
            for (int k = 0; k < cols; k++) {
                T left_val = mat[i_off + k];
                if (left_val == Semiring::add_id()) continue;  // 零元优化

                int k_off = k * other.cols;
                for (int j = 0; j < other.cols; j++) {
                    T product = Semiring::mul(left_val, other.mat[k_off + j]);
                    res.mat[i_res_off + j] = Semiring::add(res.mat[i_res_off + j], product);
                }
            }
        }
        return res;
    }

    Matrix& operator*=(const Matrix& other) {
        *this = *this * other;
        return *this;
    }
```

```
    Matrix qpow(long long b) const {
        assert(rows == cols);
        Matrix res = Matrix::identity(rows);
        Matrix base = *this;
        while (b > 0) {
            if (b & 1) res = res * base;
            base = base * base;
            b >>= 1;
        }
        return res;
    }
};
```

数论\rand

数论\rand\rand.cpp

```
std::mt19937_64 rng(std::chrono::steady_clock::now().time_since_epoch().count());
i64 rnd(i64 l, i64 r) {
    assert(l < r && "⚠ 随机数错误: 区间 [l, r) 必须满足 l < r");
    return std::uniform_int_distribution<i64>(l, r - 1)(rng);
}
```

数论\位运算

数论\位运算\bits.cpp

```cpp
int msb_diff(i64 l, i64 r) { return 63 - __builtin_clzll(l ^ r); }
int high_bit(i64 x) { return x ? 63 - __builtin_clzll(x) : -1; }
int low_bit(i64 x) { return x ? __builtin_ctzll(x) : 64; }
int popcount(i64 x) { return __builtin_popcountll(x); }
i64 lowbit_val(i64 x) { return x & -x; }
bool is_pow2(i64 x) { return x > 0 && !(x & (x - 1)); }
i64 ceil_pow2(i64 x) { return x <= 1 ? 1 : 1LL << (64 - __builtin_clzll(x - 1)); }
/*
注：所有“第几位”均指【从右往左数】（0-based），即二进制最低位(LSB)为第 0 位。

msb_diff(l, r);    // 返回 l 和 r 二进制最高位不同的是第几位。
high_bit(x);       // 返回 x 最高位 1 的位置。等价于 floor(log2(x))。x=0 返回 -1。
low_bit(x);        // 返回 x 最低位 1 的位置。x=0 返回 64。
popcount(x);       // 返回二进制中 1 的个数。
lowbit_val(x);     // 返回 x 最低位 1 代表的数值 (即 2^low_bit(x))。
is_pow2(x);        // 判断是否是 2 的幂 (1, 2, 4, 8...)。
ceil_pow2(x);      // 返回 >= x 的最小的 2 的幂。
*/
```

数论\线性基

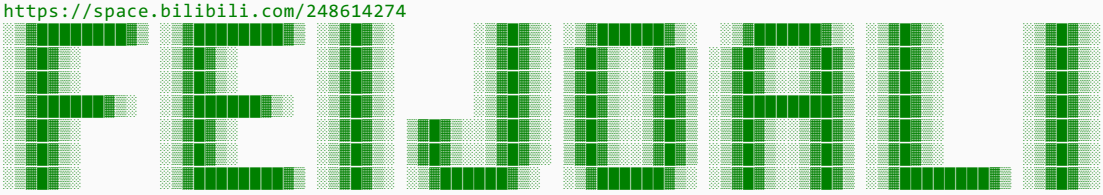
数论\线性基\线性基.cpp

```cpp
struct LinearBasis {
    constexpr static int BITS = 60; // 适配 long long，如果是 int 范围可改 31
    std::array<int, BITS> basis;

    void init() {
        std::fill(basis.begin(), basis.end(), 0); // 多组数据记得清空
    }

    void insert(int x) {
        for (int i = BITS - 1; i >= 0; i--) {
            if (!(x & (1LL << i))) continue; // 第 i 位为 0，跳过
            if (!basis[i]) { basis[i] = x; return; } // 该位无基底，插入并结束
            x ^= basis[i]; // 该位有基底，进行异或消元
        }
    }

    int query_max() {
        int res = 0;
        for (int i = BITS - 1; i >= 0; i--)
            res = std::max(res, res ^ basis[i]); // 贪心：如果异或后变大就选
        return res;
    }

    int query_min() {
        for (int i = 0; i < BITS; i++)
            if (basis[i]) return basis[i]; // 最低位的基底就是最小非零异或值
        return 0; // 线性基为空
    }

    bool check(int x) {
        for (int i = BITS - 1; i >= 0; i--) {
            if (!(x & (1LL << i))) continue; // 不需要消去该位
            if (!basis[i]) return false; // 需要消但没有对应基底，无法表示
            x ^= basis[i]; // 消元
        }
        return true; // 能被消成 0 说明可以表示
    }
} lb;
/*
lb.init();          // 初始化/清空线性基 (多组数据必调)
lb.insert(x);       // 插入元素 x
lb.query_max();     // 查询能异或出的最大值
lb.query_min();     // 查询能异或出的最小非零值
lb.check(x);        // 检查 x 是否能被线性基表示 (true:能)
*/
```

根目录

Head.cpp

```cpp
#include "bits/stdc++.h"
__attribute__((constructor)) void Feijoa_Li() { std::ios::sync_with_stdio(false), std::cin.tie(nullptr); }
/*
关注b站<Feijoa_Li>谢谢喵
UID:248614274
https://space.bilibili.com/248614274
```



```cpp
 */
using u128 = unsigned __int128;
using i128 = __int128;

using u64 = unsigned long long;
using i64 = long long;

using u32 = unsigned int;
using i32 = int;

#define dbg std::cout << "line : " << __LINE__ << " "
#define int long long

constexpr i64 inf = 1e18L;
constexpr i64 MOD = 1e9 + 7;
/*-----------------------------------------------------------------------------------------------------*/

/*

*/

void slu() {
    ;
    return;
}

i32 main() {
    int t = 1;
    std::cin >> t;

    while (t--) slu();
    return 0;
}
```

计算几何

计算几何\complex.cpp

```cpp
// ===== 点/向量类型 =====
using Point = complex<double>;
// 表示二维点 (x,y)，例如 Point(3,4)

// 获取横纵坐标（注意是函数，不是成员变量）
#define x real
#define y imag

// ====== 基本运算 ======

// 点积：a·b = |a||b|cosθ
double dot(Point a, Point b) {
    return (conj(a) * b).x();
}

// 叉积：a×b = |a||b|sinθ
// 几何意义：>0 表示 b 在 a 的逆时针方向，<0 表示顺时针
double cross(Point a, Point b) {
    return (conj(a) * b).y();
}

// 向量长度 ||a||
double norm(Point a) {
    return abs(a);
}

// 向量长度平方（避免开方，更快）
double norm2(Point a) {
    return dot(a, a);
}

// 向量旋转 θ 弧度
Point rotate(Point a, double theta) {
    return a * polar(1.0, theta);
}

// 将向量 a 单位化（长度 = 1）
Point unit(Point a) {
    return a / abs(a);
}

// 两点间距离
double dist(Point a, Point b) {
    return abs(a - b);
}

// 两向量夹角 [0, π]
double angle(Point a, Point b) {
    return acos(dot(a, b) / (abs(a) * abs(b)));
}

// 向量的极角 [-π, π]
double argAngle(Point a) {
    return arg(a); // atan2(a.y(), a.x())
}

// 将点 p 投影到向量 ab 上
Point projection(Point a, Point b, Point p) {
    Point v = b - a, w = p - a;
    return a + v * (dot(w, v) / norm2(v));
}

// 求点 p 关于直线 ab 的对称点
Point reflection(Point a, Point b, Point p) {
    return projection(a, b, p) * 2.0 - p;
}

// 判断点 c 是否在 ab 直线上（允许误差 eps）
bool onLine(Point a, Point b, Point c, double eps = 1e-9) {
    return fabs(cross(b - a, c - a)) < eps;
}

// 判断点 c 是否在线段 ab 上
bool onSegment(Point a, Point b, Point c, double eps = 1e-9) {
    return onLine(a, b, c, eps) &&
           dot(c - a, c - b) <= eps;
}

// 向量的象限 (极角排序辅助函数)
int quadrant(Point p) {
```

```
    if (p.y() > 0 || (p.y() == 0 && p.x() >= 0)) return 0; // 上半平面
    return 1; // 下半平面
}

// ====== 凸包 (Monotone Chain) ======
// 输入：点集 a
// 输出：凸包点集（逆时针顺序，不含重复点）
vector<Point> complexHull(vector<Point> a) {
    // 按 x 再按 y 排序
    sort(a.begin(), a.end(), [&](Point p, Point q) {
        if (p.x() != q.x()) return p.x() < q.x();
        return p.y() < q.y();
    });

    vector<Point> l, h;
    for (auto p : a) {
        // 下凸壳
        while (l.size() > 1 && cross(l.back() - l[l.size()-2], p - l.back()) <= 0)
            l.pop_back();
        l.push_back(p);

        // 上凸壳
        while (h.size() > 1 && cross(h.back() - h[h.size()-2], p - h.back()) >= 0)
            h.pop_back();
        h.push_back(p);
    }

    // 拼接：h 逆序 + l 去掉首尾
    reverse(h.begin(), h.end());
    h.insert(h.end(), l.begin()+1, l.end()-1);

    return h; // 凸包点集，按逆时针顺序
}
```

计算几何\tmp.cpp

```cpp
using Real = double;
const Real EPS = 1e-9;
const Real PI = acos(-1.0);

// 符号函数：处理精度
template <typename T>
int sign(T x) {
    if constexpr (std::is_floating_point_v<T>) {
        if (std::abs(x) < EPS) return 0;
    }
    return x < 0 ? -1 : 1;
}

template <typename T>
int dcmp(T x, T y) {
    return sign(x - y);
}

template <typename T>
struct Point {
    T x, y;

    constexpr Point() : x(0), y(0) {}
    constexpr Point(T x, T y) : x(x), y(y) {}

    // 基础运算
    constexpr Point operator+(const Point &r) const { return {x + r.x, y + r.y}; }
    constexpr Point operator-(const Point &r) const { return {x - r.x, y - r.y}; }
    constexpr Point operator*(T k) const { return {x * k, y * k}; }
    constexpr Point operator/(T k) const { return {x / k, y / k}; }

    constexpr Point &operator+=(const Point &r) { x += r.x; y += r.y; return *this; }
    constexpr Point &operator-=(const Point &r) { x -= r.x; y -= r.y; return *this; }
    constexpr Point &operator*=(T k) { x *= k; y *= k; return *this; }
    constexpr Point &operator/=(T k) { x /= k; y /= k; return *this; }

    // 比较运算 (用于排序或Map)
    bool operator<(const Point &r) const {
        int s = dcmp(x, r.x);
        return s == 0 ? dcmp(y, r.y) < 0 : s < 0;
    }
    bool operator==(const Point &r) const { return dcmp(x, r.x) == 0 && dcmp(y, r.y) == 0; }
    bool operator!=(const Point &r) const { return !(*this == r); }

    // 几何属性
    T abs2() const { return x * x + y * y; } // 模长的平方
    double abs() const { return std::sqrt(abs2()); } // 模长
    Point unit() const { return *this / abs(); } // 单位向量
    Point normal() const { return {-y, x}; } // 法向量（逆时针旋转90度）

    // 旋转 (逆时针 rad)
    Point rotate(double rad) const {
        return {x * cos(rad) - y * sin(rad), x * sin(rad) + y * cos(rad)};
    }

    // 点积与叉积
    friend T dot(const Point &a, const Point &b) { return a.x * b.x + a.y * b.y; }
    friend T cross(const Point &a, const Point &b) { return a.x * b.y - a.y * b.x; }
    // 两个向量的夹角 A·B = |A||B|cos(theta)
    friend double angle(const Point &a, const Point &b) {
        return acos(std::clamp<double>(dot(a, b) / a.abs() / b.abs(), -1.0, 1.0));
    }

    // 输入输出
    friend std::istream &operator>>(std::istream &is, Point &p) { return is >> p.x >> p.y; }
    friend std::ostream &operator<<(std::ostream &os, const Point &p) { return os << p.x << " " << p.y; }
};



// 直线 (点向式: p + t*v)
template <typename T>
struct Line {
    Point<T> p, v;

    constexpr Line() {}
    constexpr Line(Point<T> p, Point<T> v) : p(p), v(v) {}

    // 两点构造直线
    static Line fromPoints(Point<T> a, Point<T> b) { return {a, b - a}; }
```

```cpp
    // 点是否在直线上
    bool onLine(Point<T> a) const { return sign(cross(v, a - p)) == 0; }

    // 点到直线的距离
    double dist(Point<T> a) const { return std::abs(cross(v, a - p)) / v.abs(); }

    // 点在直线上的投影点
    Point<T> proj(Point<T> a) const { return p + v * (dot(v, a - p) / v.abs2()); }

    // 点关于直线的对称点
    Point<T> reflect(Point<T> a) const { return proj(a) * 2 - a; }
};

using L = Line<Real>;

// ----- 常用几何算法函数库 -----

// 1. 线段相交判定 (包含端点)
// 快速排斥 + 跨立实验
template<typename T>
bool onSegment(Point<T> p, Point<T> a, Point<T> b) {
    return sign(cross(a - p, b - p)) == 0 && sign(dot(a - p, b - p)) <= 0;
}

template<typename T>
bool segIntersect(Point<T> a1, Point<T> a2, Point<T> b1, Point<T> b2) {
    T c1 = cross(a2 - a1, b1 - a1), c2 = cross(a2 - a1, b2 - a1);
    T c3 = cross(b2 - b1, a1 - b1), c4 = cross(b2 - b1, a2 - b1);
    if (sign(c1) * sign(c2) < 0 && sign(c3) * sign(c4) < 0) return true;
    // 处理端点重合或共线情况
    if (onSegment(b1, a1, a2)) return true;
    if (onSegment(b2, a1, a2)) return true;
    if (onSegment(a1, b1, b2)) return true;
    if (onSegment(a2, b1, b2)) return true;
    return false;
}

// 2. 直线交点 (调用前需保证不平行: cross(v1, v2) != 0)
// Returns P + v * t
template<typename T>
Point<T> lineIntersection(Point<T> p1, Point<T> v1, Point<T> p2, Point<T> v2) {
    T t = cross(p2 - p1, v2) / cross(v1, v2);
    return p1 + v1 * t;
}

// 3. 多边形面积 (鞋带公式)
template<typename T>
double polygonArea(const std::vector<Point<T>>& poly) {
    double area = 0;
    int n = poly.size();
    for (int i = 0; i < n; i++) {
        area += cross(poly[i], poly[(i + 1) % n]);
    }
    return std::abs(area) / 2.0;
}

// 4. 点在多边形内判定 (射线法)
// 2: 内部, 1: 边界, 0: 外部
template<typename T>
int pointInPoly(Point<T> p, const std::vector<Point<T>>& poly) {
    int n = poly.size();
    int wn = 0;
    for (int i = 0; i < n; i++) {
        Point<T> p1 = poly[i], p2 = poly[(i + 1) % n];
        if (onSegment(p, p1, p2)) return 1; // 边界
        int k = sign(cross(p2 - p1, p - p1));
        int d1 = sign(p1.y - p.y);
        int d2 = sign(p2.y - p.y);
        if (k > 0 && d1 <= 0 && d2 > 0) wn++;
        if (k < 0 && d2 <= 0 && d1 > 0) wn--;
    }
    return wn != 0 ? 2 : 0;
}
using P = Point<long long>; // 或 double
/* 使用说明:
P a(1, 2), b(3, 4);
a + b, a - b;        // 向量加减
a * 2.5, a / 2;      // 数乘除
dot(a, b);           // 点积
cross(a, b);         // 叉积
a.abs();             // 模长
a.unit();            // 单位向量
a.rotate(PI/2);      // 旋转
```

```
Line<double> l = Line<double>::fromPoints(a, b);
l.proj(p);           // 点 p 在直线 l 上的投影
l.dist(p);           // 点 p 到直线 l 的距离

onSegment(p, a, b);        // 点是否在线段 ab 上
segIntersect(a, b, c, d);  // 线段 ab 与 cd 是否相交
lineIntersection(p1, v1, p2, v2); // 求直线交点
polygonArea(pts);          // 多边形面积
*/
```

计算几何\静态凸包_with_complex.cpp

```cpp
using Point = std::complex<i64>;

#define x real
#define y imag

auto dot(const Point &a, const Point &b) {
    return (std::conj(a) * b).x();
}

auto cross(const Point &a, const Point &b) {
    return (std::conj(a) * b).y();
}

auto rot(const Point &p) {
    return Point(-p.y(), p.x());
}

auto complexHull(std::vector<Point> a) {
    std::sort(a.begin(), a.end(), [&](auto a, auto b) {
        if (a.x() != b.x()) {
            return a.x() < b.x();
        } else {
            return a.y() < b.y();
        }
    });

    std::vector<Point> l, h;

    for (auto p : a) {
        while (l.size() > 1 && cross(l.back() - l[l.size() - 2], p - l.back()) <= 0) {
            l.pop_back();
        }

        while (h.size() > 1 && cross(h.back() - h[h.size() - 2], p - h.back()) >= 0) {
            h.pop_back();
        }

        l.push_back(p);
        h.push_back(p);
    }

    std::reverse(h.begin(), h.end());

    h.insert(h.end(), l.begin() + 1, l.end() - 1);

    return h;
}

int sgn(Point p) {
    if (p.y() > 0 || (p.y() == 0 && p.x() < 0)) {
        return 0;
    } else {
        return 1;
    }
}
```